# Algebra and Discrete Mathematics
# ADM_A_B

doc. Xiaolu Hou, PhD.

FIIT, STU
xiaolu.hou @ stuba.sk

# Course Outline

- Vectors and matrices
- System of linear equations
- Matrix inverse and determinants
- Vector spaces, bases and matrix transformations
- Quiz one
- Matrix transformations, fundamental spaces and decompositions
- Eulerian tours and Hamiltonian cycles
- Shortest Paths, Project Scheduling, Trees, and Networks
- Quiz two
- Matching
- Make-up Quizzes

# Recommended reading

- Saoub, K. R. (2017). A tour through graph theory. Chapman and Hall/CRC.
  - Sections 1.1 – 1.5, 2.1 – 2.3
  - Accessible online (free copy)
  - Alternative download link

# Lecture outline

- Definitions and terminologies
- Different types of graphs
- Touring a graph
- Eulerian circuit algorithms
- Eulerization
- Existence of Hamiltonian cycles
- Traveling salesman problem
- Digraphs and Asymmetric Traveling Salesman Problem

# Eulerian tours and Hamiltonian cycles

- Definitions and terminologies
- Different types of graphs
- Touring a graph
- Eulerian circuit algorithms
- Eulerization
- Existence of Hamiltonian cycles
- Traveling salesman problem
- Digraphs and Asymmetric Traveling Salesman Problem

# Graph

**Definition**

A *graph* consists of two sets: $V(G)$, called the *vertex set*, and $E(G)$, called the edge set. An *edge*, denoted $xy$, is an unordered pair of vertices $x, y \in V(G)$, where $x$ and $y$ may be the same.

- We will often use $G$ or $G = (V, E)$ as short-hand.
- $xy$ and $yx$ are treated equally, though it is customary to write them in alphabetical order
- Later in the course: *directed graphs* – the order in which an edge is written provides additional meaning.

# Graph – example

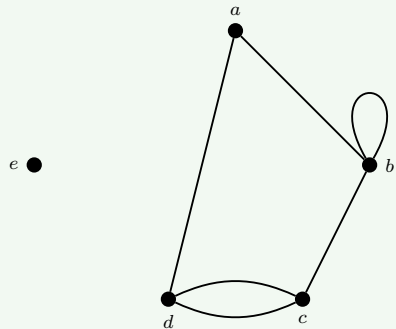
## Example

- Let $G$ be a graph where

$$V(G) = \{a, b, c, d, e\},$$

$$E(G) = \{ab, ad, bc, cd, cd, bb\}$$

- Visualization of $G$: a dot represents a vertex and an edge is a line connecting the two dots (vertices)
- a loop at $b$; no edges at $e$; two occurrences of $cd$ represent two distinct edges connecting $c$ and $d$



# Terminologies

## Definition

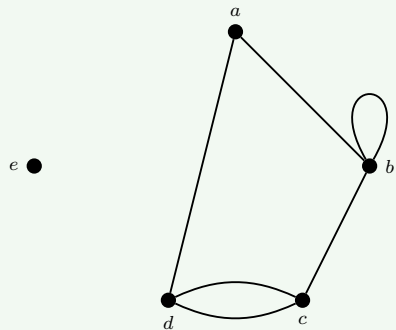
Let $G = (V, E)$ be a graph

- If $xy \in E$, then $x$ and $y$ are the *endpoints* of the edge $xy$. Each of $x$ and $y$ is *incident to* the edge $xy$.
- *Adjacent* edges: share an endpoint
- *Adjacent vertices* (*neighbors*): incident to the same edge
- $N(v)$: the set of all neighbors of a vertex $v$
- *Isolated vertex*: not incident to any edge
- *Loop*: both endpoints of an edge are the same vertex
- *Multi-edges*: two or more edges with the same endpoints
- *Degree* of a vertex: number of edges incident to the vertex, where a loop contributes $2$. Denoted $\deg(v)$. If $\deg(v)$ is even (resp. odd), then $v$ is called an *even* (resp. *odd*) vertex.

# Graph – example

## Example

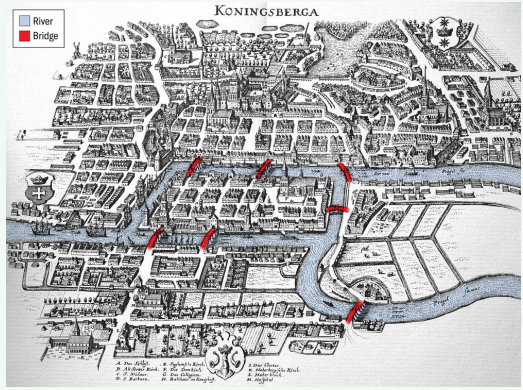
- Edges $ab$ and $ad$ are adjacent
- $a$ and $b$ are adjacent vertices
- $N(d) = \{a, c\}$, $N(b) = \{a, b, c\}$
- $e$ is an isolated vertex
- $bb$ is a loop
- $cd$ is a multi-edge
- $\deg(a) = 2$, $\deg(b) = 4$, $\deg(c) = 3$, $\deg(d) = 3$, $\deg(e) = 0$



# Modeling with a graph – Königsberg bridge problem

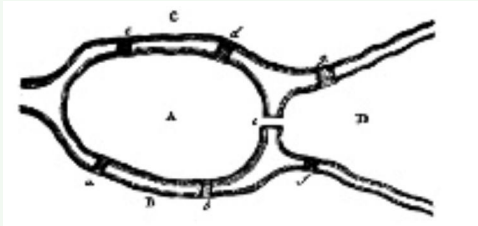
## Example

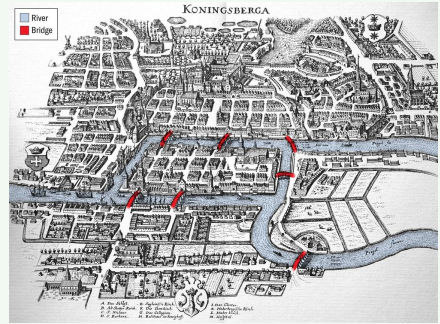
- In 1736, Leonhard Euler published a short paper on the bridges of Königsberg.
- Can you leave your home, travel across each of the bridges in the city exactly once, and then return home?
- The puzzle is described as the birth of graph theory



# Modeling with a graph – Königsberg bridge problem

## Example

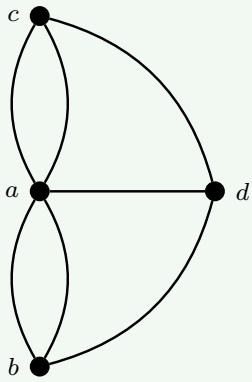


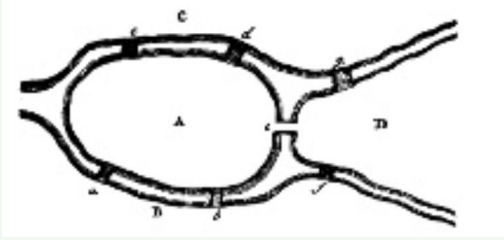




Euler reduced the map of Königsberg to a simpler version where only the relationships between landmasses were of importance

# Modeling with a graph – Königsberg bridge problem

## Example



We model the city by a graph in which the vertices represent the island, the south bank, the north bank, and the peninsula ($a, b, c, d$, respectively), and the edges represent the bridges. Can you start at a vertex, traverse each edge exactly once, and return to your starting point?

# Order and size

**Definition**

- A graph $G = (V(G), E(G))$ is called *finite* if both $V(G)$ and $E(G)$ are finite.
- A graph that is not finite is called an *infinite* graph
- The *order* of $G$ is the number of vertices of $G$, i.e. $|V(G)|$.
- The *size* of $G$ is the number of edges of $G$, i.e. $|E(G)|$.

# Eulerian tours and Hamiltonian cycles

- Definitions and terminologies
- Different types of graphs
- Touring a graph
- Eulerian circuit algorithms
- Eulerization
- Existence of Hamiltonian cycles
- Traveling salesman problem
- Digraphs and Asymmetric Traveling Salesman Problem

# Simple graph

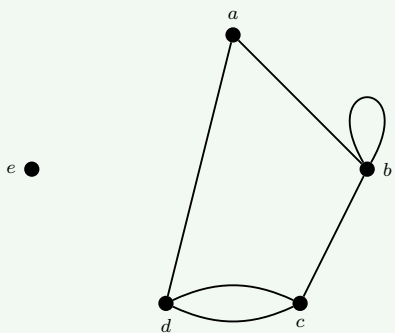
**Definition**

- *Pseudograph*: can contain multi-edges and loops
- *Multigraph*: can contain multi-edges, but no loops
- *Simple graph*: no multi-edges or loops
- *Trivial graph*: one vertex, no edges

# Graphs – example

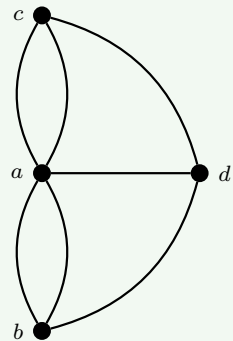
## Example

Pseudograph




Order: $5$, size: $6$

Multigraph



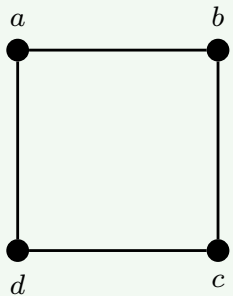

Order: $4$, size: $7$

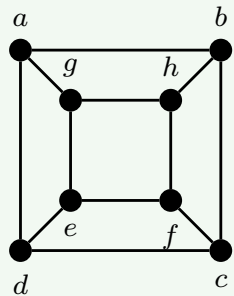
# Regular graph

## Definition

- *Regular graph*: all of its vertices have the same degree
- $k$-*regular*: all vertices have degree $k$
- A 3-regular graph is also called a *cubic* graph

## Example



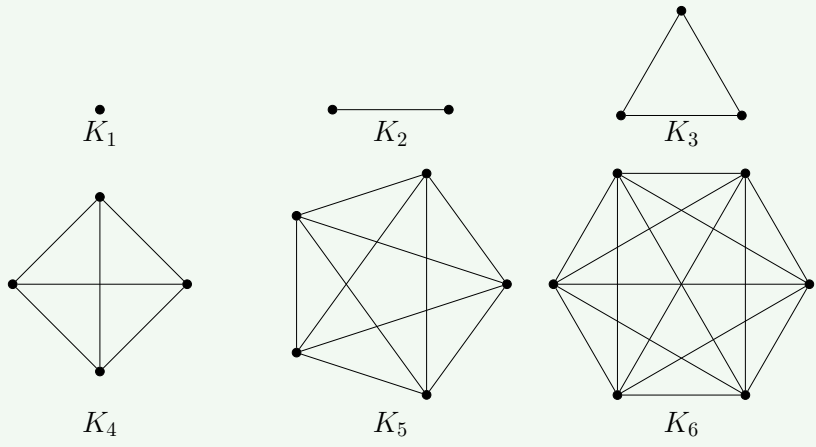
# Complete graph

**Definition**

A simple graph $G$ is *complete* if every pair of distinct vertices is adjacent. The complete graph on $n$ vertices is denoted $K_n$.

# The first six complete graphs

**Example**



# Complement of a graph

## Definition

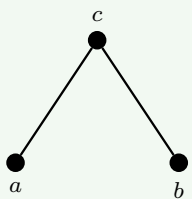
Let $G = (V, E)$ be a simple graph. The *complement* of $G$, denoted $\overline{G} = (V, \overline{E})$, is the graph with the same vertex set as $G$, where

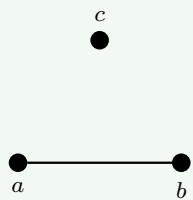
$$uv \in \overline{E} \quad \text{if and only if} \quad uv \notin E$$

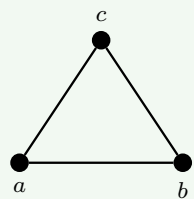
for all distinct vertices $u, v \in V$.

- $G$ and $\overline{G}$ together form a complete graph.

## Example



# Bipartite graphs

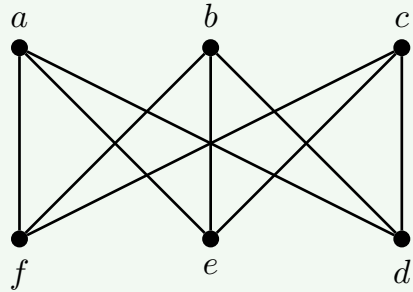
## Definition

A graph $G = (V, E)$ is *bipartite* if the vertices can be partitioned into two sets, $V_1$ and $V_2$, so that

$$V_1 \cap V_2 = \emptyset, \quad V = V_1 \cup V_2,$$

and every edge has exactly one endpoint in $V_1$ and the other endpoint in $V_2$.

## Example



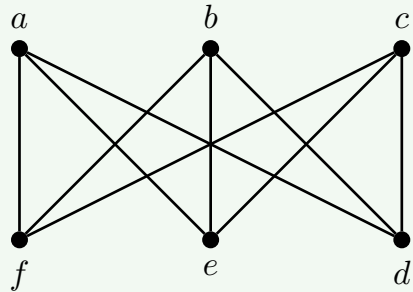
# Complete bipartite graph

## Definition

A simple bipartite graph $G = (V_1 \cup V_2, E)$ is a *complete bipartite graph* if every vertex in $V_1$ is adjacent to every vertex in $V_2$. If $|V_1| = m$ and $|V_2| = n$, we write $K_{m,n}$.

## Example

$K_{3,3}$



# Eulerian tours and Hamiltonian cycles

- Definitions and terminologies
- Different types of graphs
- Touring a graph
- Eulerian circuit algorithms
- Eulerization
- Existence of Hamiltonian cycles
- Traveling salesman problem
- Digraphs and Asymmetric Traveling Salesman Problem

# Definitions

## Definition

Let $G$ be a graph

- *Walk*: a sequence of vertices so that there is an edge between consecutive vertices. A walk can repeat vertices and edges.
- *Trail*: a walk with no repeated edges
- *Path*: a trail with no repeated vertices. A path on $n$ vertices is denoted $P_n$.
- *Closed walk*: a walk that starts and ends at the same vertex
- *Circuit*: a closed trail
- *Cycle*: a closed path. A cycle on $n$ vertices is denoted $C_n$.

*Length* of any of these tours is the number of edges.

- Path $\subseteq$ Trail $\subseteq$ Walk
- Cycle $\subseteq$ Circuit $\subseteq$ Closed walk

# Length of a tour – example

**Example**

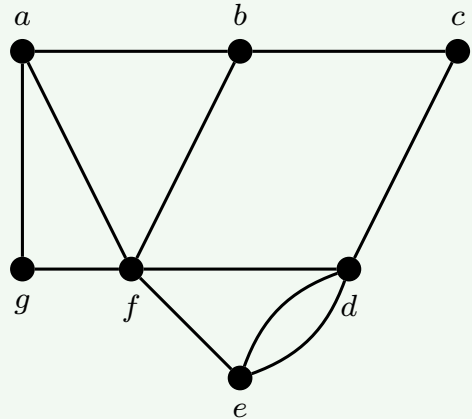
- $P_n$ has length $n - 1$
- $C_n$ has length $n$

# Touring a graph – example

## Example

Given the graph, find

- a trail (that is not a path) from $a$ to $c$
- a path from $a$ to $c$
- a circuit (that is not a cycle) starting at $b$
- a cycle starting at $b$



# Touring a graph – example

## Example

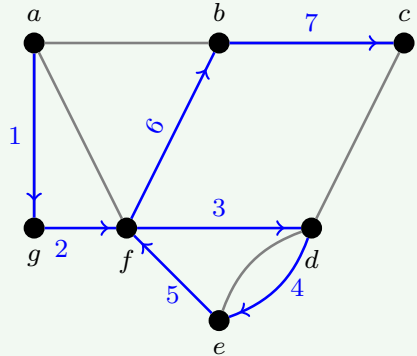



Figure: A trail from $a$ to $c$

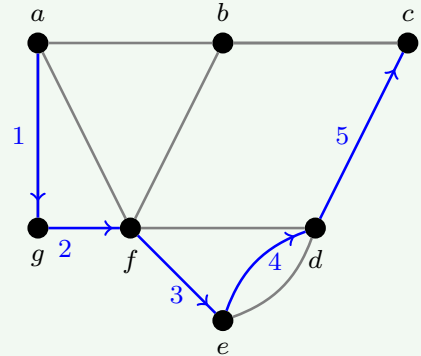



Figure: A path from $a$ to $c$

# Touring a graph – example

## Example

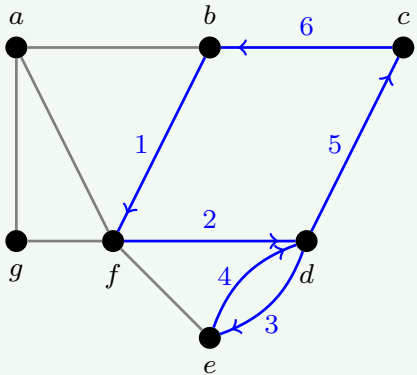



Figure: A circuit starting at $b$

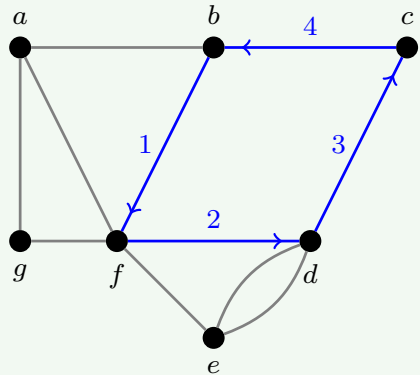



Figure: A cycle starting at $b$

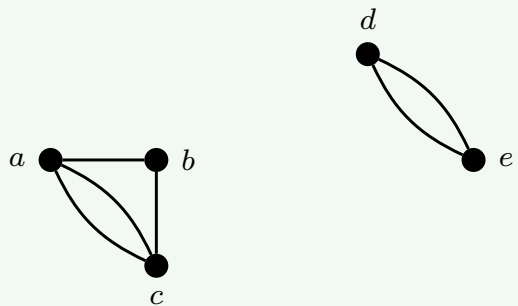
# Connected vertices

## Definition

Let $G$ be a graph. Two vertices $x$ and $y$ are *connected* if there exists a path from $x$ to $y$ in $G$. The graph $G$ is *connected* if every pair of distinct vertices is connected.

## Example

Not connected



# Eulerian circuit

## Definition

Let $G$ be a graph. An *Eulerian circuit* is a circuit that uses every edge of $G$ exactly once. An *Eulerian trail* is an open[a] trail that uses every edge of $G$ exactly once. If a *connected* graph $G$ contains an Eulerian circuit, then $G$ is called *Eulerian*. If a *connected* graph $G$ contains an Eulerian trail but not an Eulerian circuit, then $G$ is called *semi-Eulerian*.

[a] not a circuit

## Note

A graph may contain an Eulerian circuit (or trail) even if it is not connected, provided that all vertices with nonzero degree belong to the same connected component. In this course, we require Eulerian and semi-Eulerian graphs to be connected.

# Eulerian graph

**Theorem**

*A graph $G$ is Eulerian if and only if*

- *$G$ is connected and*
- *every vertex has an even degree*

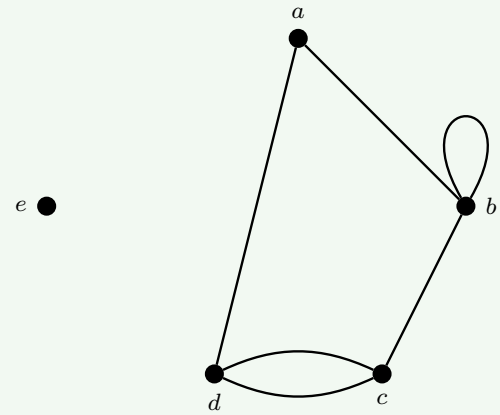
*A graph $G$ is semi-Eulerian if and only if*

- *$G$ is connected and*
- *exactly two vertices have odd degree*

- When traveling through a graph, we need to pair each entry edge with an exit edge.
- If a vertex is odd, then there is no complete pairing of incident edges, so we would eventually get stuck at that vertex.
- If the starting and ending locations are different, then exactly two vertices must be odd: the starting vertex has one extra exit edge, and the ending vertex has one extra entry edge.

# Eulerian and semi-Eulerian graphs – example

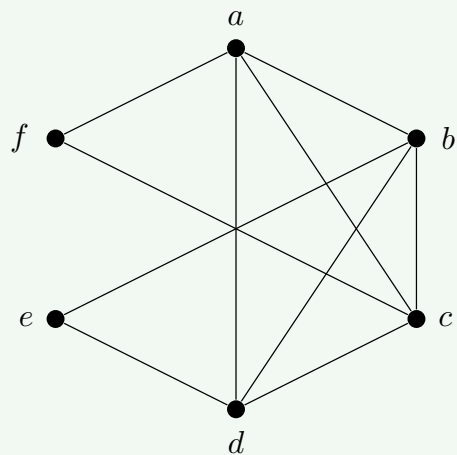
## Example




Not connected – not Eulerian, not semi-Eulerian. However, the graph contains an Eulerian trail, since the only disconnected vertex is isolated. The graph has two *components*.

# Eulerian and semi-Eulerian graphs – example

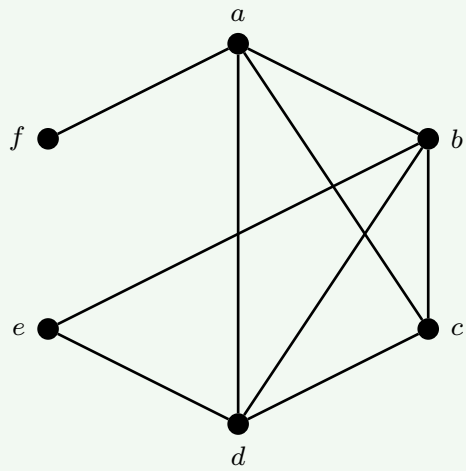
Example




Eulerian – all vertices have degree $2$ or $4$

# Eulerian and semi-Eulerian graphs – example

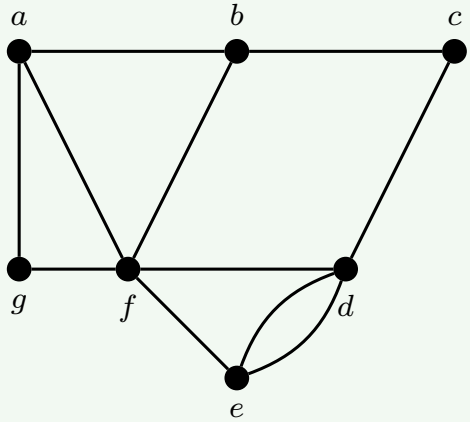
Example




semi-Eulerian – exactly two vertices are odd

# Eulerian and semi-Eulerian graphs – example

## Example




Connected, but not Eulerian or semi-Eulerian - more than two odd vertices $\{a, b, e, f\}$

# Handshaking Lemma

**Theorem (Handshaking Lemma)**

*Let $G = (V, E)$ be a finite graph. Suppose $V = \{v_1, v_2, \ldots, v_n\}$, then*

$$\deg(v_1) + \deg(v_2) + \cdots + \deg(v_n) = 2|E|$$

**Proof.**

Since each edge contributes two to the sum of degrees, one at each endpoint (or two at the same vertex in the case of a loop), the sum of degrees is even and equal to twice the number of edges. □

# Corollary

**Corollary**

*In any finite graph, there is an even number of odd vertices.*

**Proof.**

$$\sum_{v_i \text{ has odd degree}} \deg(v_i) + \sum_{v_j \text{ has even degree}} \deg(v_j) = 2|E|$$

The second term on the left is even, and the total sum is even. Thus, the sum of the degrees of the odd vertices must also be even. Since a sum of odd integers is even if and only if there is an even number of summands, the number of odd vertices is even. □

# Eulerian tours and Hamiltonian cycles

- Definitions and terminologies
- Different types of graphs
- Touring a graph
- Eulerian circuit algorithms
- Eulerization
- Existence of Hamiltonian cycles
- Traveling salesman problem
- Digraphs and Asymmetric Traveling Salesman Problem

# Fleury's Algorithm

- Find an Eulerian circuit or an Eulerian trail.
- By the previous theorem, the input graph must be connected and have either zero or two odd vertices.

# Fleury's Algorithm

1. Choose a starting vertex:
   - If the graph $G$ has no vertices of odd degree, start from any vertex
   - If $G$ has exactly two vertices of odd degree, start from one of them
2. Choose an edge to traverse: Select an edge $vw$ incident to the current vertex $v$ such that, after removing it, $v$ and $w$ are still connected, unless no other choice is available.
3. Traverse and remove the edge: Move to the other endpoint of the selected edge, then delete the edge from the graph. The new vertex becomes the current vertex.
4. Repeat Steps 2 and 3 until there are no more edges left in the graph.

Output:

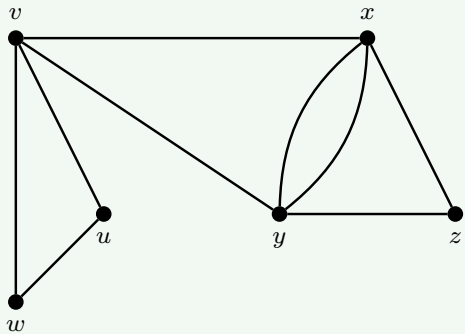
- If $G$ originally had no odd-degree vertices, Eulerian circuit
- If $G$ had exactly two odd-degree vertices, Eulerian trail

Practical implementation: Maintain two copies of the graph:

- One for tracking the Eulerian trail or circuit.
- Another for dynamically removing edges

# Fleury's Algorithm – example

## Example



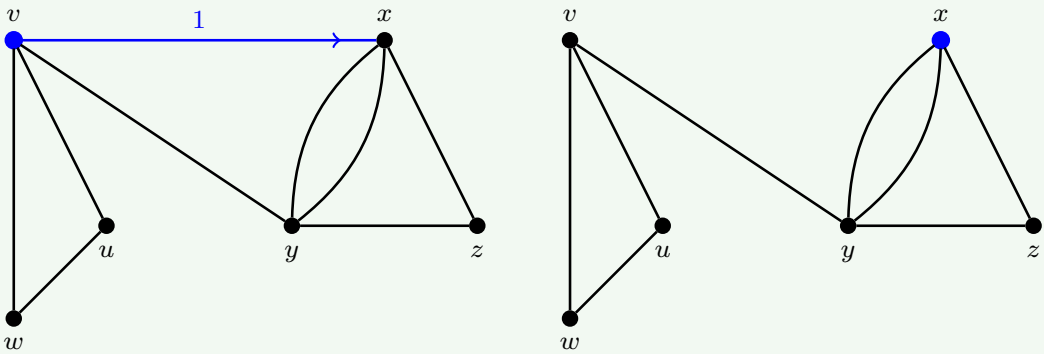

The graph is Eulerian:

$\deg(u) = 2, \quad \deg(v) = 4, \quad \deg(w) = 2, \quad \deg(x) = 4, \quad \deg(y) = 4, \quad \deg(z) = 2$

- Step 1. Start at vertex $v$.
- Step 2. Choose edge $vx$.

# Fleury's Algorithm – example

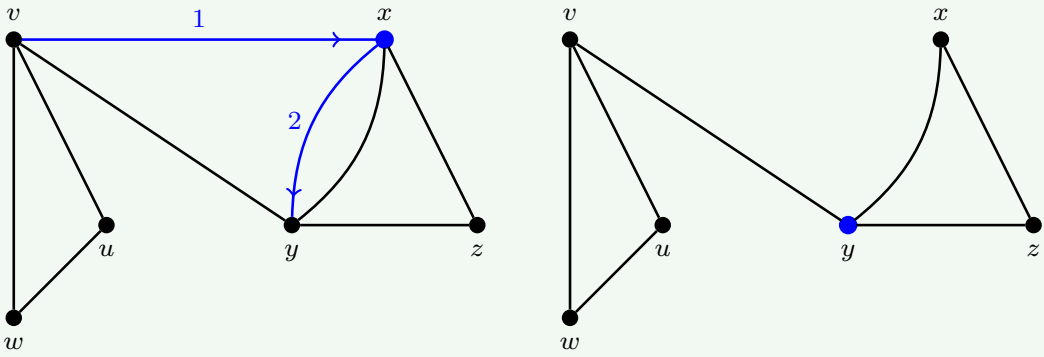
## Example




- Step 3. Travel to vertex $x$ and delete edge $vx$. Current vertex: $x$.

# Fleury's Algorithm – example

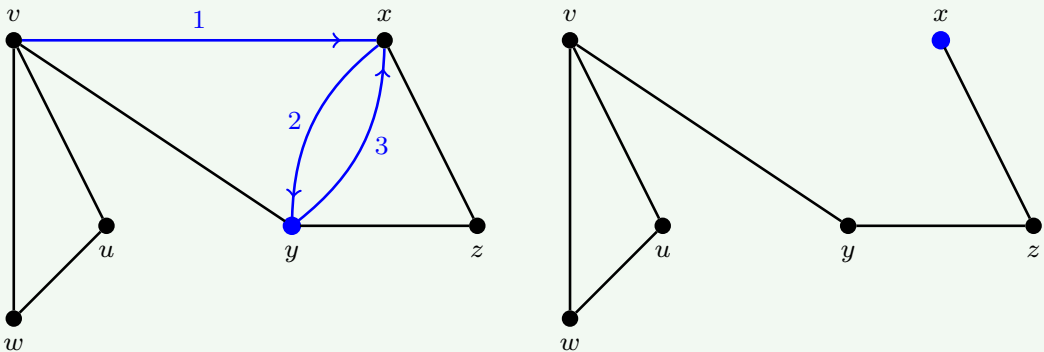
## Example




- Step 2. Choose edge $xy$.
- Step 3. Travel to vertex $y$ and delete edge $xy$. Current vertex: $y$.

# Fleury's Algorithm – example

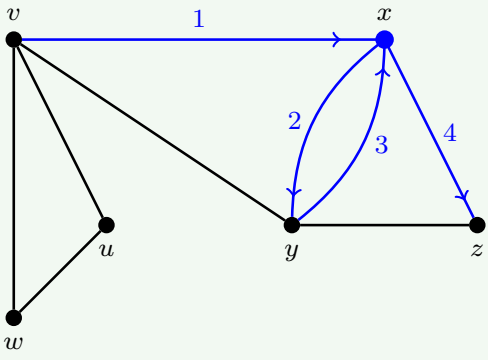
## Example

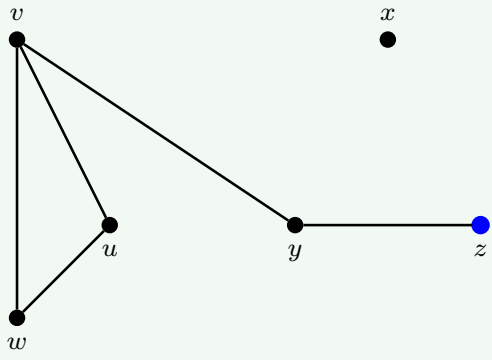



- Step 2. Choose edge $yx$. Note: we cannot choose edge $yv$, since its removal would disconnect the remaining graph.
- Step 3. Travel to vertex $x$ and delete edge $yx$. Current vertex: $x$.

# Fleury's Algorithm – example

## Example

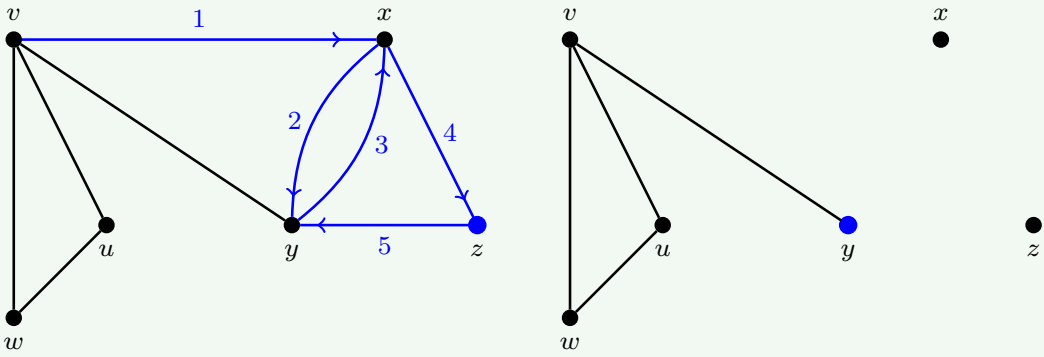



- Step 2. The only available edge is $xz$.
- Step 3. Travel to vertex $z$ and delete edge $xz$. Current vertex: $z$.

# Fleury's Algorithm – example

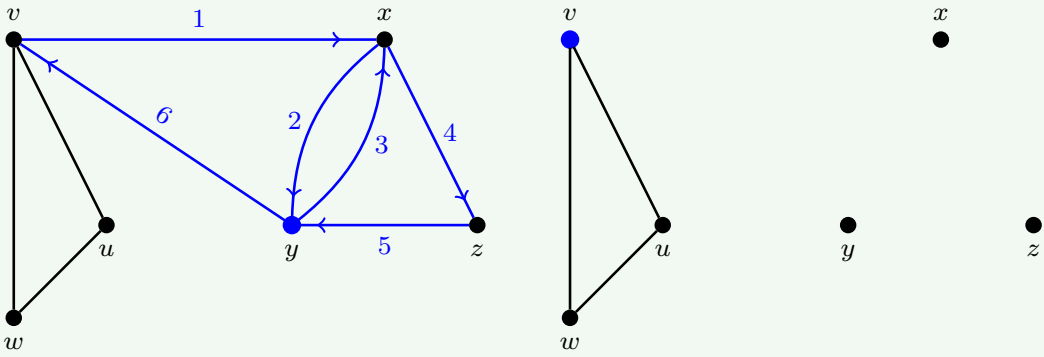
## Example




- Step 2. The only available edge is $zy$.
- Step 3. Travel to vertex $y$ and delete edge $zy$. Current vertex: $y$.

# Fleury's Algorithm – example

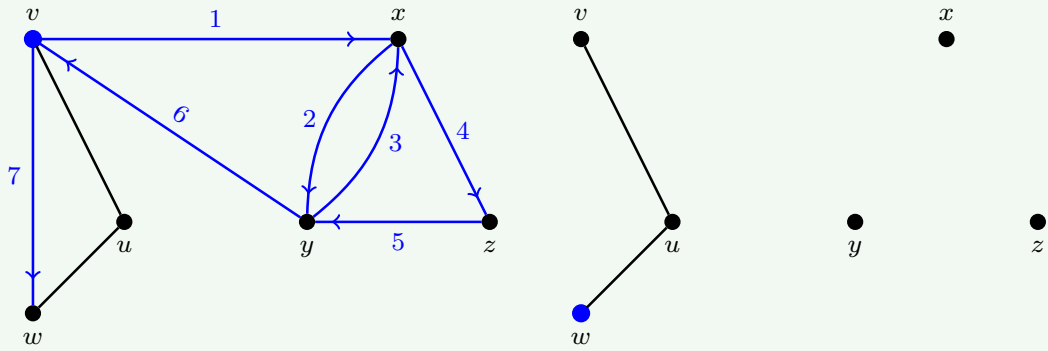
## Example




- Step 2. The only available edge is $yv$.
- Step 3. Travel to vertex $v$ and delete edge $yv$. Current vertex: $v$.

# Fleury's Algorithm – example

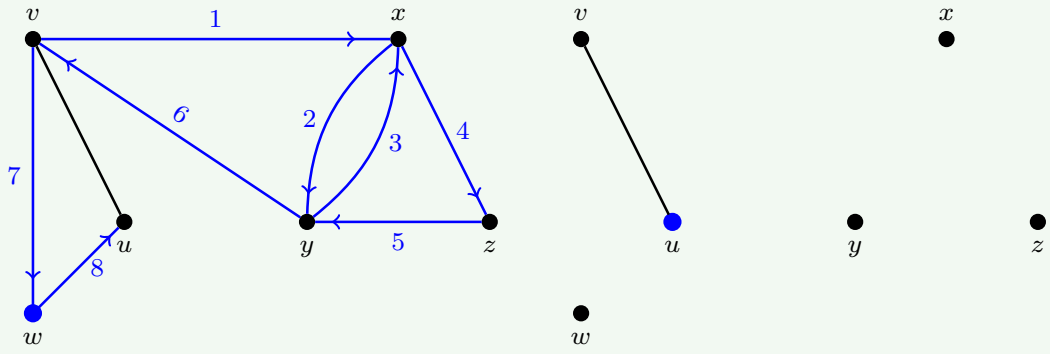
## Example




- Step 2. Both $vu$ and $vw$ can be chosen. Here, we choose $vw$.
- Step 3. Travel to vertex $w$ and delete edge $vw$. Current vertex: $w$.

# Fleury's Algorithm – example

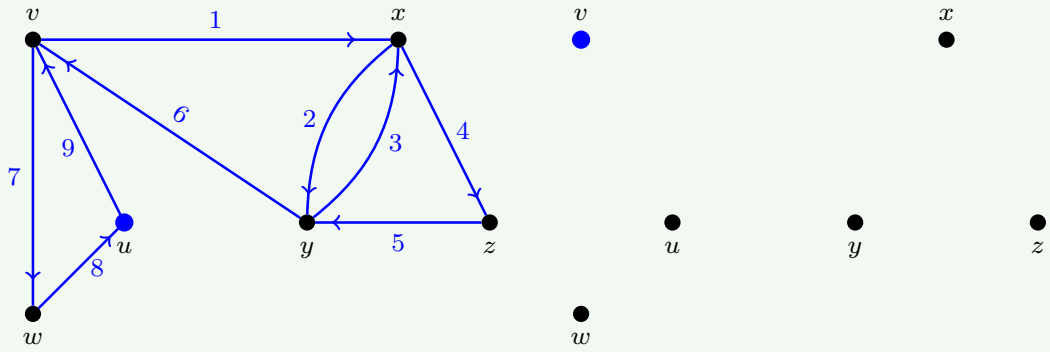
## Example




- Step 2. The only available edge is $wu$.
- Step 3. Travel to vertex $u$ and delete edge $wu$. Current vertex: $u$.

# Fleury's Algorithm – example

## Example




- Step 2. The only available edge is $uv$.
- Step 3. Travel to vertex $v$ and delete edge $uv$.
- We have obtained an Eulerian circuit starting and ending at vertex $v$.

# Hierholzer's Algorithm

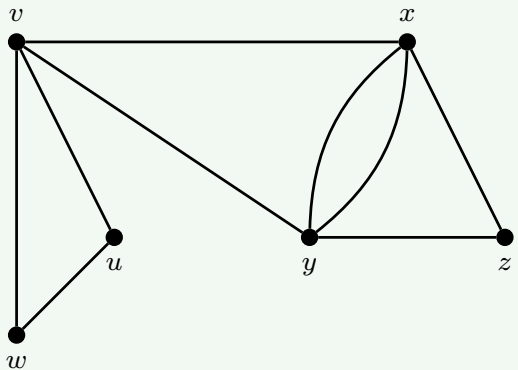
- Find an Eulerian circuit
- Input: an Eulerian graph $G$ – connected, all vertices are even

# Hierholzer's Algorithm

1. Choose a starting vertex, say $v$, and find an initial circuit $C$ starting at $v$.
2. If there exists a vertex $x$ in the current circuit $C$ that has unused edges, form a new circuit $C'$ starting and ending at $x$, using only unused edges.
3. Merge the newly found circuit $C'$ into the existing circuit $C$, updating $C$ accordingly.
4. Repeat steps 2 and 3 until all edges of $G$ have been traversed.

# Hierholzer's Algorithm – example

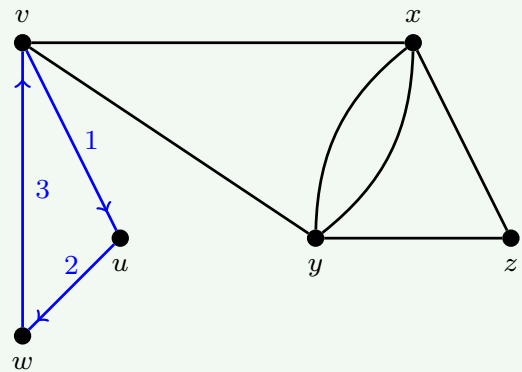
## Example




Same graph, we have discussed that the graph is Eulerian.

# Hierholzer's Algorithm – example

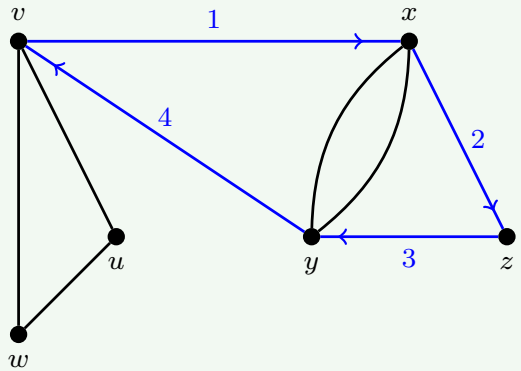
## Example




- Step 1: choose vertex $v$ and find a circuit starting at $v$

# Hierholzer's Algorithm – example

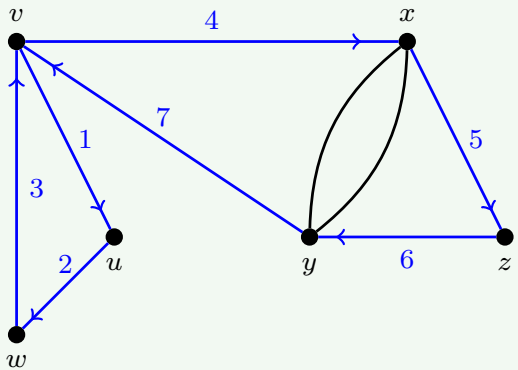
## Example




- Step 2: since $\deg(v) = 4$, two unused edges remain incident to $v$. We can find a second circuit starting at $v$.

# Hierholzer's Algorithm – example

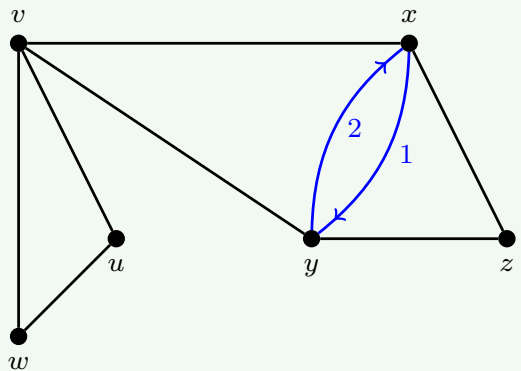
## Example




- Step 3: combine the two circuits from Step 1 and Step 2. There are multiple ways to combine two circuits.

# Hierholzer's Algorithm – example

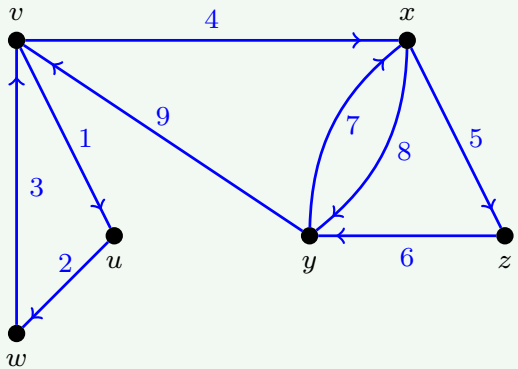
## Example




- Step 2: Since $\deg(x) = 4$, two unused edges remain incident to $x$, so we can find another circuit starting at $x$.

# Hierholzer's Algorithm – example

## Example




- Step 3: combine the new circuit with the existing one
- We have obtained an Eulerian circuit

# Eulerian tours and Hamiltonian cycles

- Definitions and terminologies
- Different types of graphs
- Touring a graph
- Eulerian circuit algorithms
- Eulerization
- Existence of Hamiltonian cycles
- Traveling salesman problem
- Digraphs and Asymmetric Traveling Salesman Problem

# Eulerization – definition

## Definition

Given a connected graph $G = (V, E)$, an *Eulerization* of $G$ is the graph $G' = (V, E')$ so that

- $G'$ is obtained by duplicating edges of $G$, and
- every vertex of $G'$ is even

A *semi-Eulerization* of $G$ results in a graph $G'$ so that

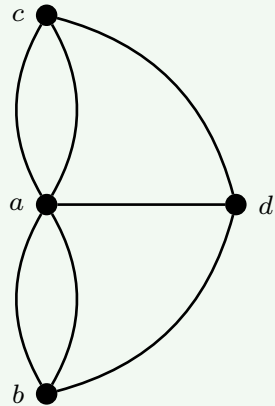
- $G'$ is obtained by duplicating edges of $G$, and
- exactly two vertices of $G'$ are odd

# Optimal exhaustive tour

- In the context of the Königsberg Bridge Problem, duplicating an edge would be equivalent to walking the same bridge twice
- Although this solution would be outside the original parameters (walking each bridge exactly once), it allows for an approximate solution using the fewest number of duplications.
- This is referred to as finding an *optimal exhaustive tour* of a graph

# The Königsberg Bridge Problem

## Example

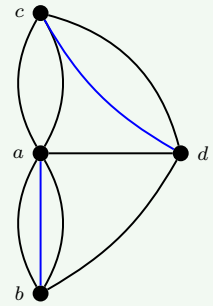



We have discussed that the graph modeling Königsberg city is not Eulerian or semi-Eulerian

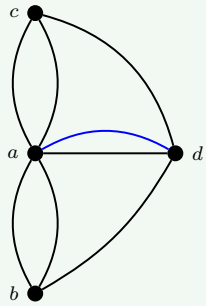
# The Königsberg Bridge problem

## Example

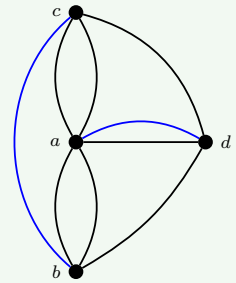
Eulerization




semi-Eulerization




not an Eulerization




- Note that when creating the new graph, edges must be duplicated, not added
- Of course edges can be added in a city, but unrealistic from the standpoint of a person touring the bridges of a city during a specific moment in time. Hence, we only allow duplications of edges

# The optimization question

- We are interested in the question: how can we Eulerize (or semi-Eulerize) a graph using the fewest number of edge duplications?
- By finding an optimal Eulerization, we obtain a closed walk of minimum length that traverses every edge of the original graph at least once.

# Eulerization method

1. Identify the odd vertices of the graph
2. Pair up the odd vertices, trying to pair as many adjacent vertices as possible while also avoiding pairing vertices that are far apart[1]
3. Duplicate the edges along an optimal path from one vertex to its pair

Note:

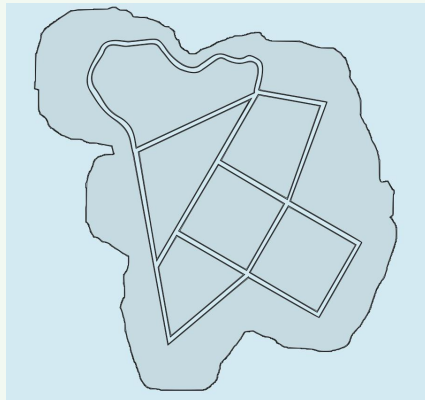
- In the process of determining which edges to duplicate along optimal paths, never repeat an edge more than once
- If an edge is crossed three times, removing two of the duplications will not change the parity of the endpoint of the edge
  - that is, a vertex will remain odd or remain even when subtracting two from the degree

[1] Distance: the length of the shortest path connecting the two vertices.

# Eulerization – example

**Example**

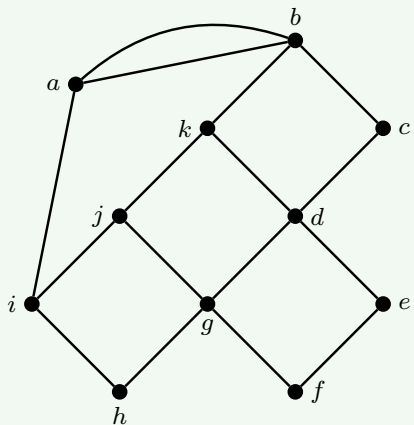
- The citizens of the small island town of Sunset Island want to hire a night patrol during the busy summer tourist season
- Model the town as a graph and find an optimal Eulerization of the graph

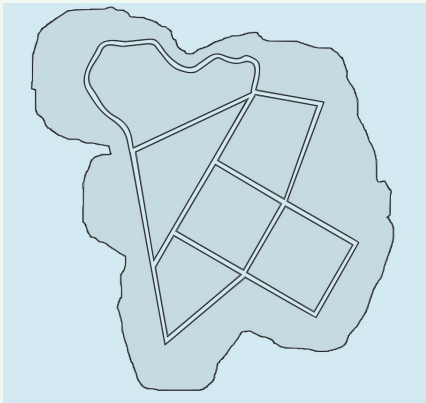


# Eulerization – example

## Example

The graph model: vertices represent intersections and edges represent street blocks.

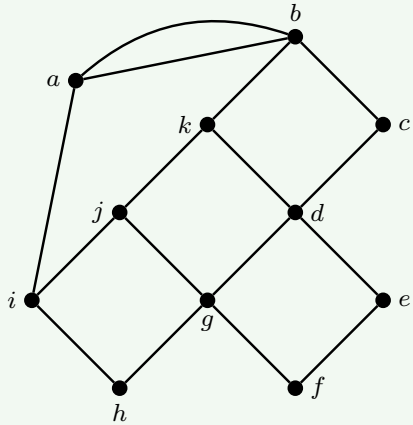


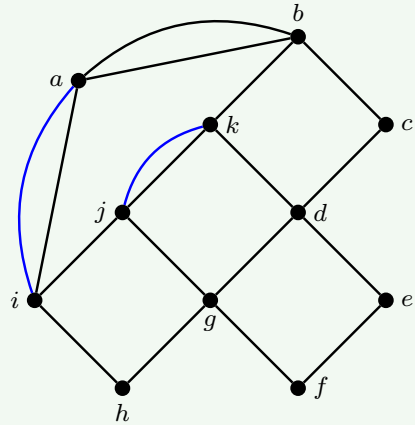



# Eulerization – example

## Example

- Four odd vertices: $a, i, j, k$
- They can be split into two pairs of adjacent vertices: $a, i$ and $j, k$

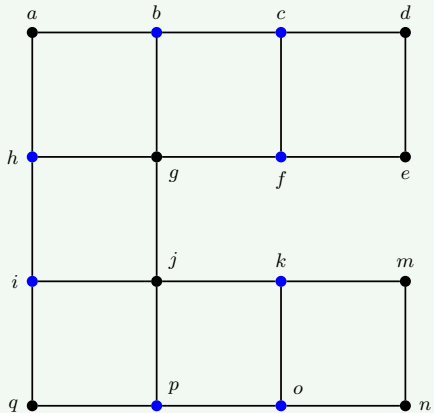


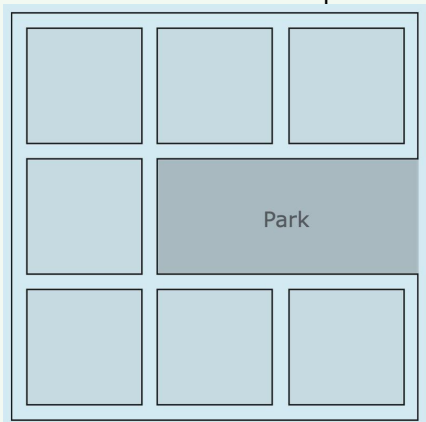



# Eulerization – example

## Example

- Task: put up fliers along each block of a small portion of a town
- Requirement: travel along each street at least once but no fliers in the park



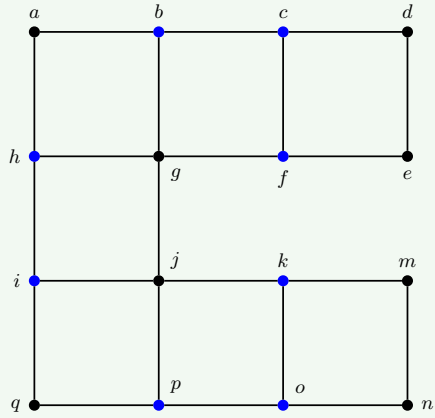




We model the graph and identify the odd vertices (in blue)

# Eulerization – example

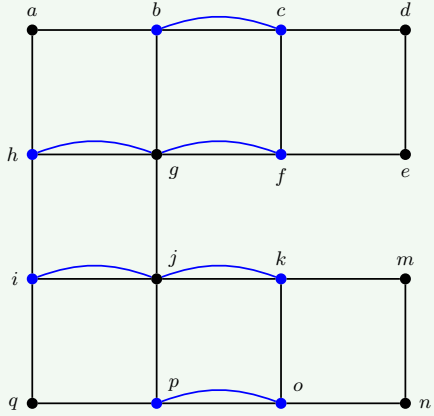
## Example

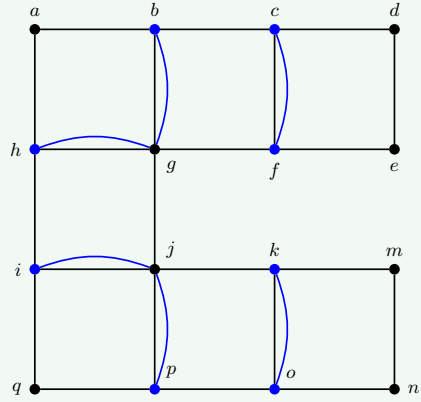
- Unlike the previous example, we cannot split all the $8$ odd vertices into adjacent pairs, which corresponds to only $4$ duplicated edges
- $5$ duplicated edges is also not possible
  - If we use at most $2$ adjacent pairs, at least $2$ pairs require paths of length at least $2$, so at least $6$ edge duplications are required.
  - If we use $3$ adjacent pairs, we are left with $1$ pair of distance $3$ apart, which requires $3$ more edges.
  - It is impossible to split all $8$ odd vertices into $4$ adjacent pairs.
- At least $6$ edge duplications are needed



# Eulerization – example

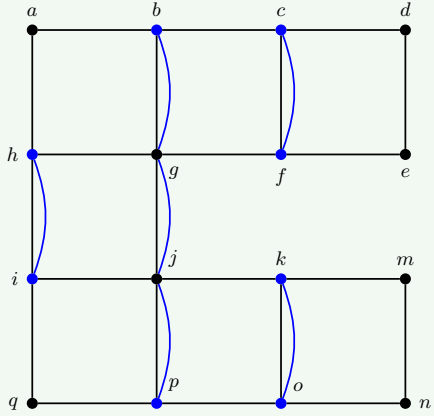
## Example

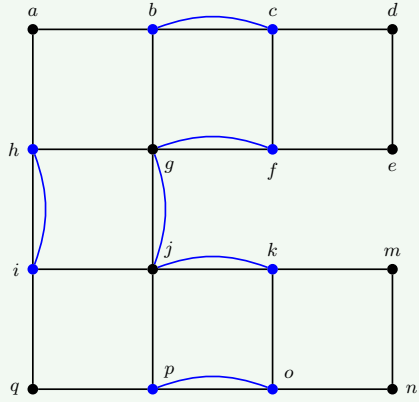


# Eulerization – example

## Example



# Weighted graph

- In the previous examples, the streets follow a highly structured grid layout, where traveling along one block is comparable to traveling along another.
- What happens when different streets require varying levels of effort to traverse?

**Definition**

A *weighted graph* $G = (V, E, \omega)$ is a graph in which each edge has a real number associated with it. This number is called the *weight* and is denoted $\omega(xy)$ for the edge $xy$.

**Remark**

- Weight of an edge can represent length, time, cost, etc.
- A weighted graph can also refer to a graph in which each of the vertices is assigned a weight

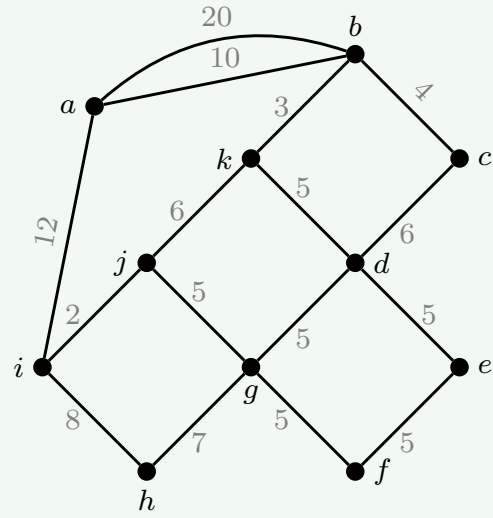
# Chinese Postman Problem

- The *Chinese Postman Problem* is to find a closed walk of minimum total weight that traverses every edge of a connected, undirected, weighted graph at least once.
- It can be solved by finding an Eulerization with minimum added weight, followed by an Eulerian circuit in the resulting graph.
- The name originates not from anything particular about postmen in China, but rather from the mathematician who first proposed the problem —the Chinese mathematician Mei-Ku Kwan (管梅谷) in 1962

# Chinese Postman Problem – example

## Example

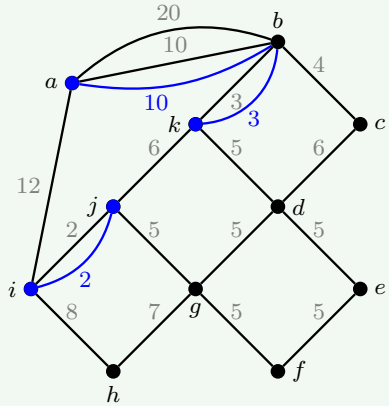
- Suppose the travel time through the streets of Sunset Island varies depending on the street.
- Find an optimal Eulerization taking into account the weights

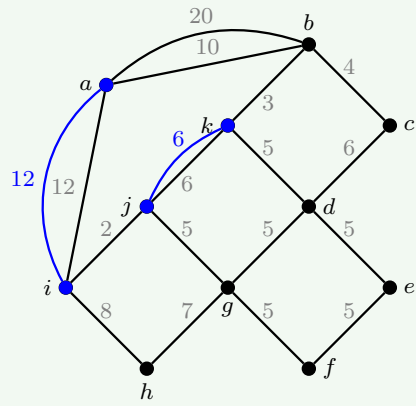


# Chinese Postman Problem – example

## Example

The previous Eulerization we have obtained (on the right) is not optimal (two duplicated edges total $18$). A better Eulerization duplicates three edges for a total cost of $15$.



# Remarks

- Be careful when duplicating edges in a graph with multi-edges, indicating which edge has been duplicated – including the edge weight clarifies which option was used.
- In general, solving the Chinese Postman Problem can be quite challenging
- Most small examples can be solved by inspection
- The choice of which edges to duplicate when working with a weighted graph relies in part on shortest paths between two vertices.
- The difficulty is in choosing which vertices to pair.
- This will be discussed later in the course

# Eulerian tours and Hamiltonian cycles

- Definitions and terminologies
- Different types of graphs
- Touring a graph
- Eulerian circuit algorithms
- Eulerization
- Existence of Hamiltonian cycles
- Traveling salesman problem
- Digraphs and Asymmetric Traveling Salesman Problem

# Hamiltonian cycle and Hamiltonian path

**Definition**

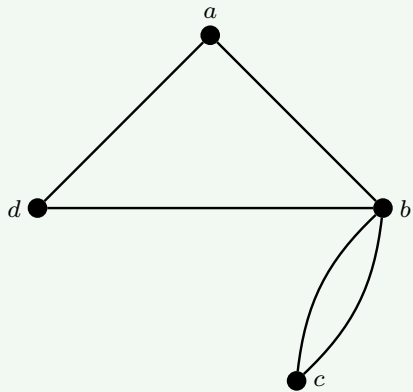
A cycle in a graph $G$ that contains every vertex of $G$ is called a *Hamiltonian cycle*. An *open*[a] path that contains every vertex is called a *Hamiltonian path*.

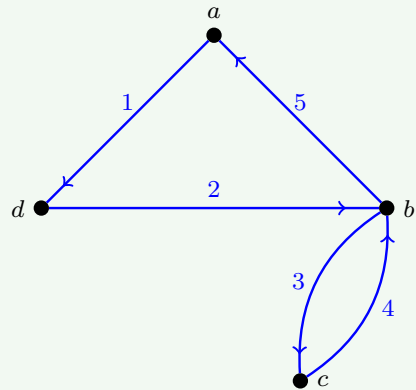
[a] not a cycle

- Recall that a cycle or a path can only pass through a vertex once, except that a cycle returns to its starting vertex. Thus, Hamiltonian cycles and paths visit every vertex exactly once, with the starting vertex repeated at the end of a Hamiltonian cycle.
- Named after mathematician: Sir William Hamilton

# Hamiltonian cycle and Eulerian circuit – example

## Example

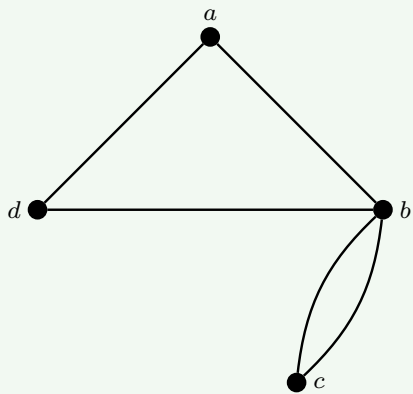


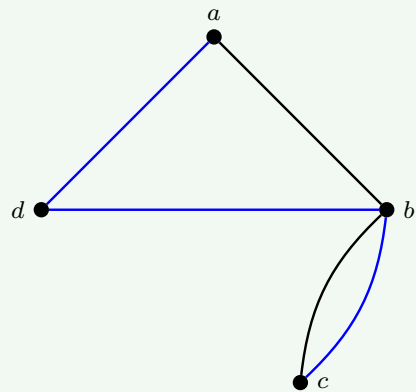




The graph is connected and all vertices are even – it contains an Eulerian circuit

# Hamiltonian cycle and Eulerian circuit – example

## Example

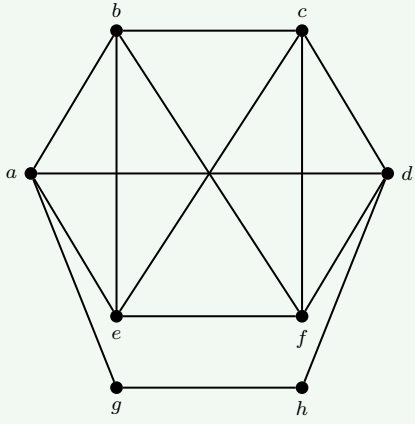


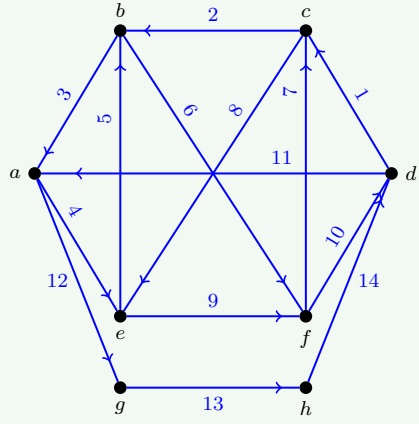




There is no Hamiltonian cycle since we need to include $c$ in the cycle and by doing so we would pass $b$ twice

# Hamiltonian cycle and Eulerian circuit – example

## Example

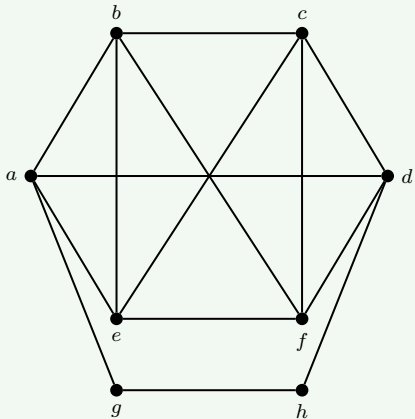


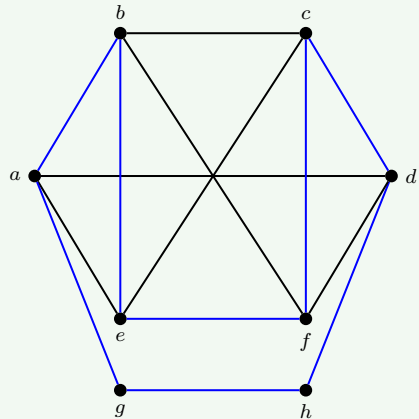




The graph is connected and all vertices are even – it contains an Eulerian circuit

# Hamiltonian cycle and Eulerian circuit – example

## Example




Hamiltonian cycles and Hamiltonian paths also exist. To find a Hamiltonian path, remove any one edge from the highlighted Hamiltonian cycle.

# Necessary conditions for Hamiltonian cycle

- $G$ must be connected
- No vertex of $G$ can have degree less than $2$
- $G$ cannot contain a cut-vertex

Cut-vertex: a vertex whose removal disconnects the graph

**Note**

These are not sufficient conditions

# A sufficient condition for Hamiltonian cycle

**Theorem (Dirac's Theorem)**

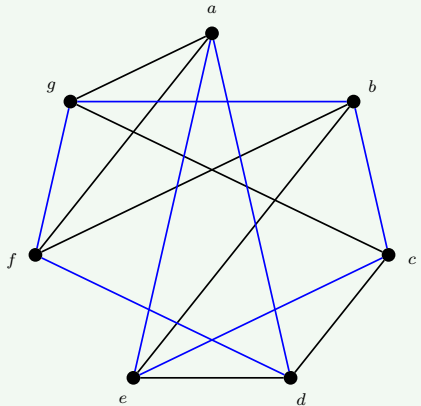
*Let $G$ be a simple graph with $n \geq 3$ vertices. If every vertex $v$ of $G$ satisfies $\deg(v) \geq \frac{n}{2}$, then $G$ has a Hamiltonian cycle.*

**Remark**

- The proof of this theorem uses the large number of neighbors of each vertex to establish the existence of a Hamiltonian cycle.
- There are several sufficient conditions for a graph to have a Hamiltonian cycle.
- Unlike Eulerian graphs, Hamiltonian graphs have no known simple characterization based on vertex degrees and connectivity.

# Existence of Hamiltonian cycle – example

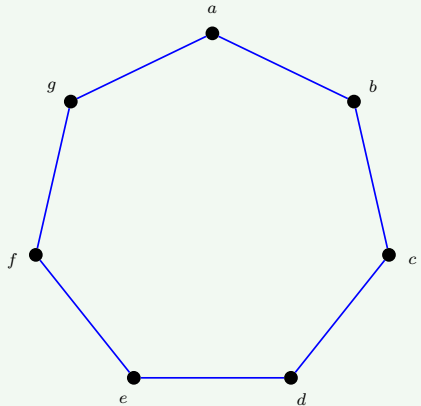
## Example




$n = 7$

$\deg = 4 \geq \frac{7}{2}$

for all vertices




$n = 7$

$\deg = 2 < \frac{7}{2}$

for all vertices

# Complete graph

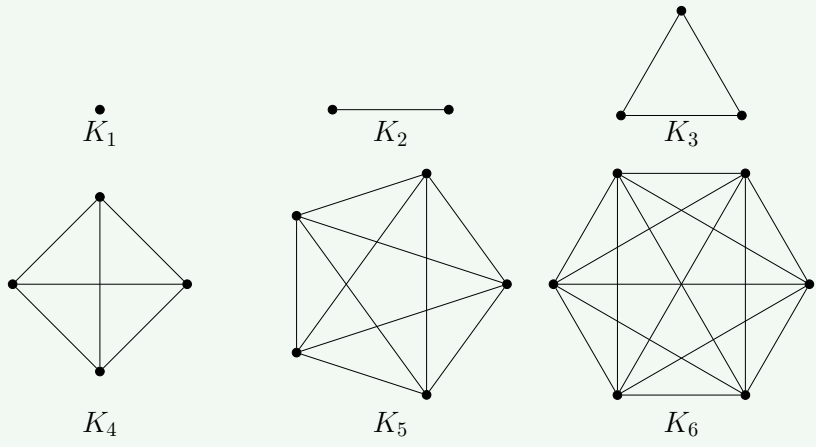
**Definition**

A simple graph $G$ is *complete* if every pair of distinct vertices is adjacent. The complete graph on $n$ vertices is denoted $K_n$.

- If we think of an edge as describing a relationship between two objects, then a complete graph represents a scenario where every pair of vertices satisfies this relationship.

# The first six complete graphs

Example



# Properties of $K_n$

- Each vertex in $K_n$ has degree $n-1$
- $K_n$ has $\frac{n(n-1)}{2}$ edges
- $K_n$ contains the most edges out of all simple graphs on $n$ vertices

**Note**

- Any complete graph (on at least $3$ vertices) satisfies the conditions of Dirac's Theorem, and therefore contains a Hamiltonian cycle

# Number of Hamiltonian cycles in $K_n$

- Fix a starting vertex as the reference point.

**Theorem**

*For $n \geq 3$, the complete graph $K_n$ has $(n-1)!$ Hamiltonian cycles starting at a fixed vertex, counting opposite traversal directions separately.*

- There are $n$ choices for the starting vertex, $n-1$ choices for the next vertex, then $n-2$, and so on.
- Hence, the number of Hamiltonian cycles starting at the fixed vertex is

$$(n-1)(n-2)\cdots 2 \cdot 1 = (n-1)!.$$

- Note: each cycle is counted twice, once in each direction.

# Eulerian tours and Hamiltonian cycles

- Definitions and terminologies
- Different types of graphs
- Touring a graph
- Eulerian circuit algorithms
- Eulerization
- Existence of Hamiltonian cycles
- Traveling salesman problem
- Digraphs and Asymmetric Traveling Salesman Problem

# The question

- How should a delivery service plan its route through a city to ensure each customer is reached?
- A traveling salesman must visit each customer city exactly once and return home while minimizing the total cost.
- Shortest Hamiltonian cycle visiting all cities in one country
- Model: weighted complete graph
- Weights: different distances
- Complete: it is reasonable to assume it's possible to travel between any two cities

# Brute Force Algorithm

- To find the Hamiltonian cycle of least total weight, one obvious method is to find all possible Hamiltonian cycles and pick the cycle with the smallest total.
- The method of trying every possibility to find an optimal solution is referred to as an *exhaustive search*, or use of the *Brute Force Algorithm*.
- Input: weighted complete graph $K_n$
- Output: minimum Hamiltonian cycle

# Brute Force Algorithm

1. Choose a starting vertex, call it $v$
2. Find all Hamiltonian cycles starting at $v$. Calculate the total weight of each cycle.
3. Compare all $(n-1)!$ cycles. Pick one with the least total weight.

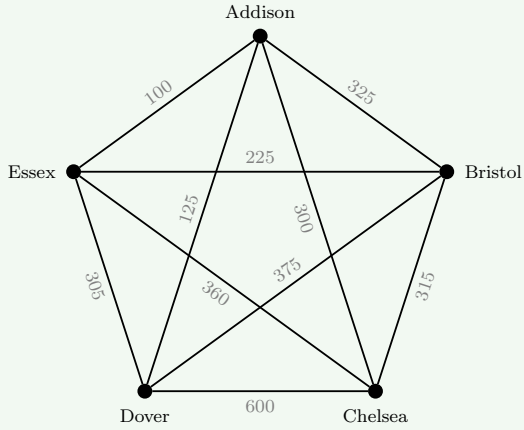
**Note**

Reversing a Hamiltonian cycle does not change its total weight.

# Brute Force Algorithm – example

## Example

Liz is planning her next business trip from her hometown of Addison and has determined the cost for travel between any of the five cities she must visit.



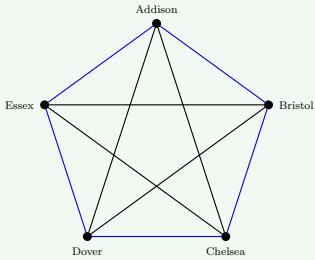
# Brute Force Algorithm – example

## Example

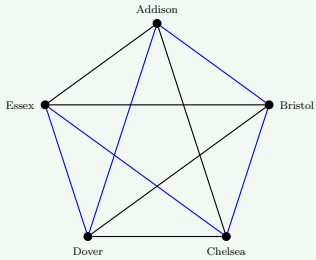
- Number of tours starting and ending at Addison: $4! = 24$.
- These represent $12$ distinct Hamiltonian cycles, since each cycle is counted twice by reversal.
- One method for finding all Hamiltonian cycles, and ensuring you indeed have all $24$, is to use alphabetical or lexicographic ordering of the cycles.
- Note that all cycles must start and end at Addison, and we will abbreviate all cities with their first letter
- For example, the first cycle is $abcdea$

# Brute Force Algorithm – example

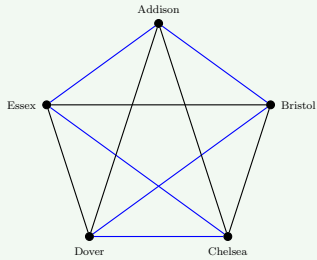
## Example




$abcdea$
$aedcba$
1645



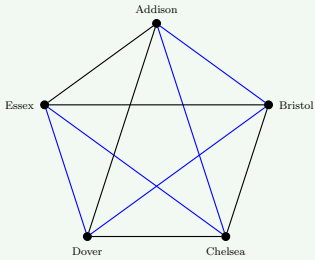

$abceda$
$adecba$
1430



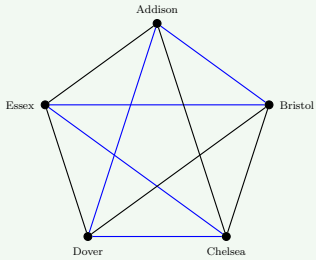

$abdcea$
$aecdba$
1760

# Brute Force Algorithm – example

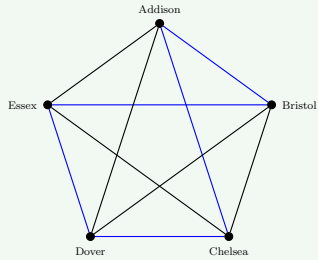
## Example




$abdeca$
$acedba$
1665



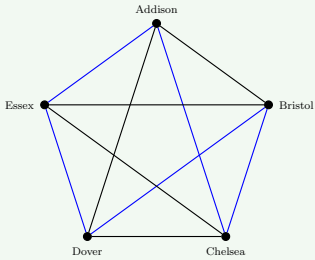

$abecda$
$adceba$
1635



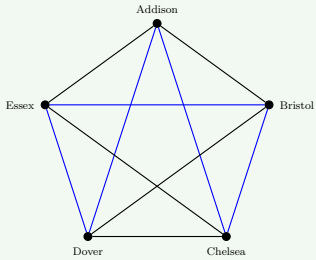

$abedca$
$acdeba$
1755

# Brute Force Algorithm – example

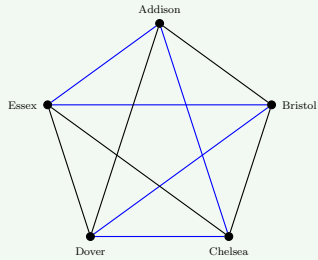
## Example




$acbdea$
$aedbca$
1395



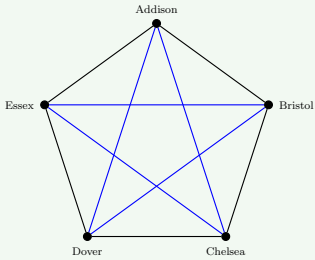

$\boldsymbol{acbeda}$
$\boldsymbol{adebca}$
1270



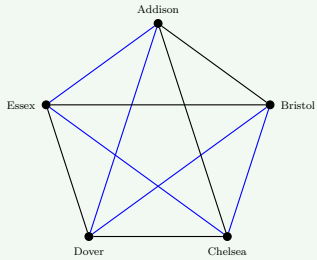

$acdbea$
$aebdca$
1600

# Brute Force Algorithm – example

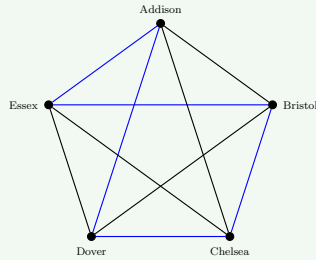
## Example




$acebda$
$adbeca$
1385




$adbcea$
$aecbda$
1275




$adcbea$
$aebcda$
1365

# Estimated effort

- Effort for using Brute Force Algorithm to find the optimal Hamiltonian cycle in $K_n$
- Best: best supercomputer
- Top $500$: top $500$ supercomputers
- Earth is about $4.54 \times 10^9$ years old

| | Supercomputers | |
|---|---|---|
| ***n*** | **Best** | **Top 500** |
| 5 | $2 \times 10^{-15}$ seconds | $2 \times 10^{-16}$ seconds |
| 15 | $2 \times 10^{-5}$ seconds | $2 \times 10^{-6}$ seconds |
| 20 | 40 seconds | 4 seconds |
| 21 | 14 minutes | 2 minutes |
| 22 | 5 hours | 32 minutes |
| 23 | 4.5 days | 12 hours |
| 24 | 16 weeks | 12 days |
| 25 | 7.5 years | 10 months |
| 26 | 2 centuries | 2 decades |
| 30 | 132 million years | 14 million years |
| 40 | $4.1 \times 10^{23}$ years | $4.3 \times 10^{22}$ years |
| 50 | $1.4 \times 10^{40}$ years | $1.5 \times 10^{39}$ years |

# Other algorithms

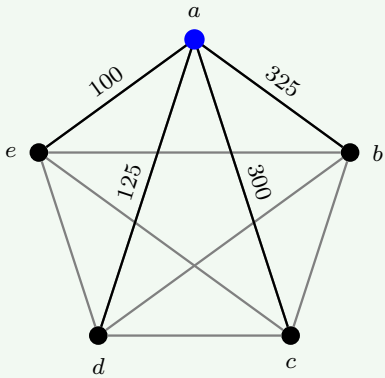
- We now discuss faster algorithms for finding good Hamiltonian cycles.
- The goal is to find an algorithm that is both efficient and optimal.
- No such algorithm is currently known for the Traveling Salesman Problem.

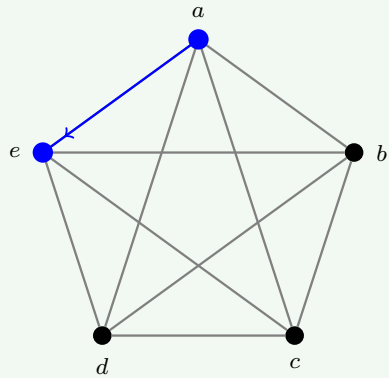
# Nearest Neighbor Algorithm

1. Choose a starting vertex, say $v$. Highlight $v$.
2. Among all edges incident to $v$, choose one with the smallest weight. If more than one edge has the same weight, choose one arbitrarily.
3. Highlight the chosen edge and move to its other endpoint $u$. Highlight $u$.
4. Repeat Steps 2 and 3, considering only edges leading to unhighlighted vertices.
5. Close the cycle by adding the edge from the last highlighted vertex back to $v$. Then calculate the total weight.

# Nearest Neighbor Algorithm – example

## Example

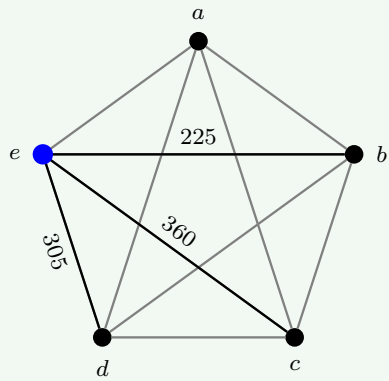


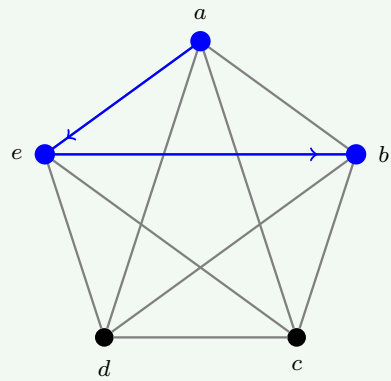




- Step 1: Starting vertex is $a$
- Step 2: Edge of smallest weight: $ae$
- Step 3: Move to vertex $e$. Highlight $e$

# Nearest Neighbor Algorithm – example

## Example

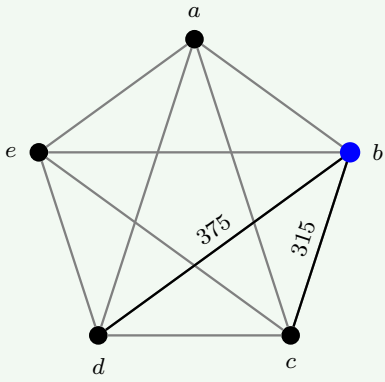


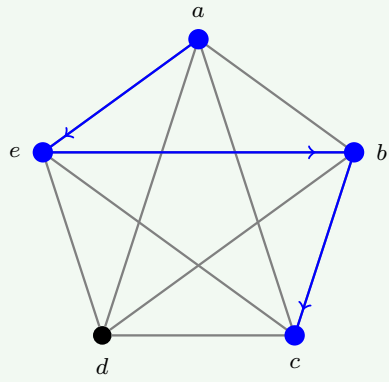




- Step 2: The edge of smallest weight is $eb$. Note that we do not consider edge $ea$.
- Step 3: Move to vertex $b$. Highlight $b$

# Nearest Neighbor Algorithm – example

## Example

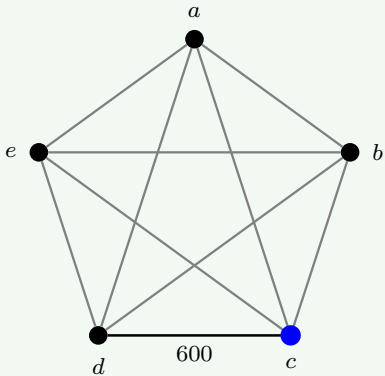


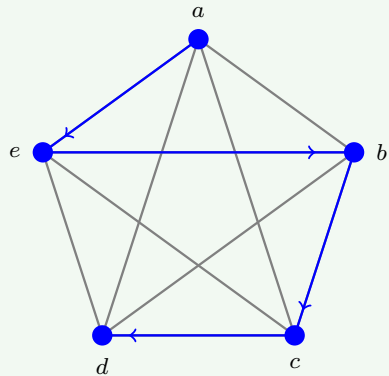




- Step 2: Edge of smallest weight: $bc$
- Step 3: Move to vertex $c$. Highlight $c$

# Nearest Neighbor Algorithm – example

## Example

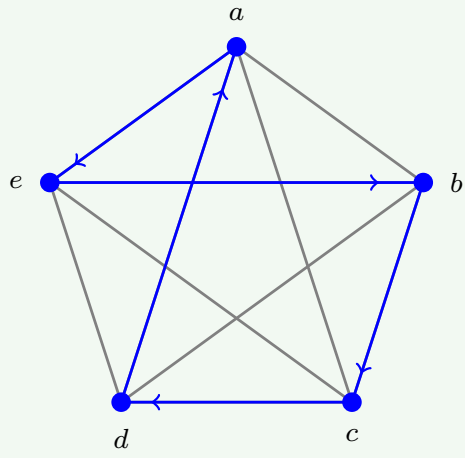



- Step 2: The only available choice is edge $cd$.
- Step 3: Move to vertex $d$. Highlight $d$

# Nearest Neighbor Algorithm – example

## Example




- Step 5: Close the cycle. Total weight is $1365$

# Remarks

- The last two edges are completely determined, since we cannot return to vertices that have already been chosen.
- This may force us to use some of the heaviest edges in the graph, as happened above.
- The arbitrary choice of a starting vertex may eliminate some light edges from consideration.
- Although we cannot avoid the first issue, we can address the second by changing the starting vertex.
- A different starting vertex may produce a different Hamiltonian cycle, which may be better or worse than the original one.

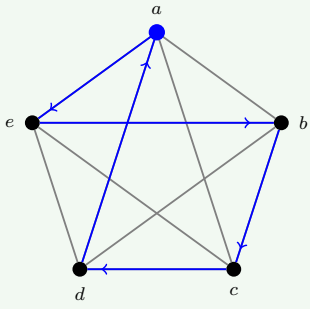
# Repetitive Nearest Neighbor Algorithm

1. Choose a starting vertex, say $v$.
2. Apply the Nearest Neighbor Algorithm.
3. Repeat Steps 1 and 2 so that each vertex of $K_n$ serves as the starting vertex.
4. Choose the cycle of least total weight and rewrite it using the desired reference point.

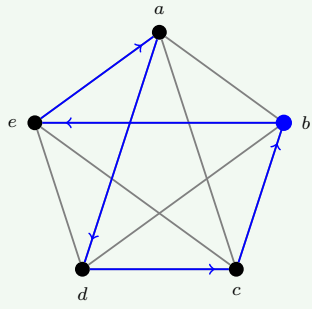
# Repetitive Nearest Neighbor Algorithm – example

## Example

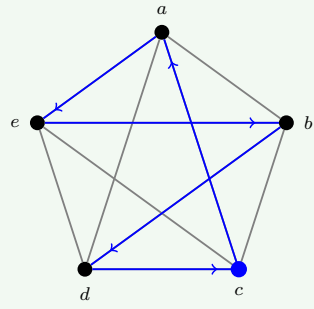
With the same example as before, the Repetitive Nearest Neighbor Algorithm gives the following cycles.




$aebcda$
$aebcda$
1365




$beadcb$
$adcbea$
1365

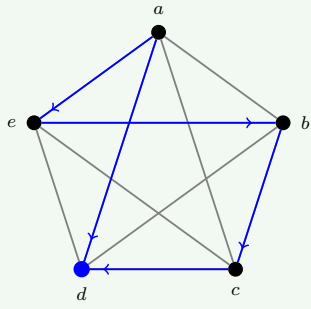



$caebdc$
$aebdca$
1600

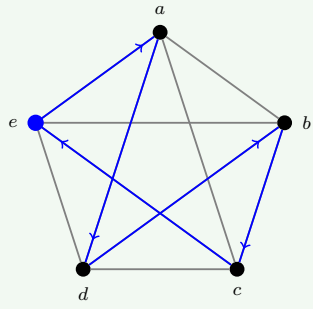
# Repetitive Nearest Neighbor Algorithm – example

## Example

With the same example as before, the Repetitive Nearest Neighbor Algorithm gives the following cycles.




$daebcd$
$aebcda$
1365




$eadbce$
$adbcea$
1275

# Remarks

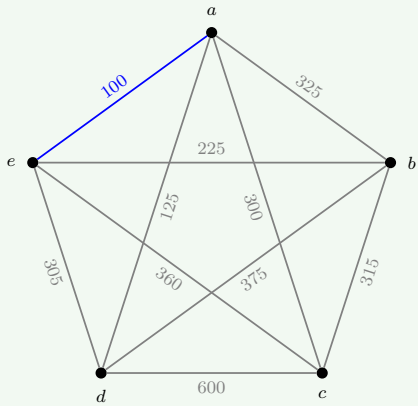
- Repetitive Nearest Neighbor often performs better than Nearest Neighbor; however, there is still no guarantee that it will produce an optimal cycle.
- Even though Repetitive Nearest Neighbor addressed the concern of missing small weight edges, it is still possible that some of these will be bypassed as we travel a cycle
- Cheapest Link Algorithm: choosing edges in order of weight as opposed to edges along a tour

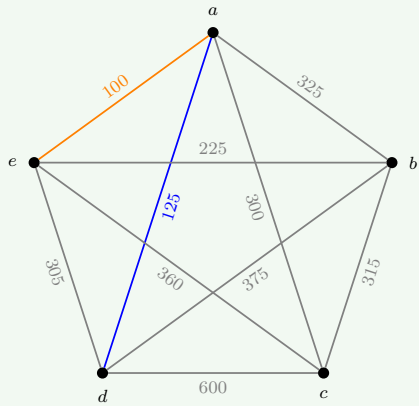
# Cheapest Link Algorithm

1. Among all edges in the graph, choose one with the smallest weight. If more than one edge has the same weight, choose one arbitrarily. Highlight the chosen edge.
2. Repeat Step 1 subject to the following conditions:
   - no vertex has three highlighted edges incident to it
   - no edge is chosen if it would create a cycle before all vertices are included
3. Calculate the total weight

# Cheapest Link Algorithm – example

## Example

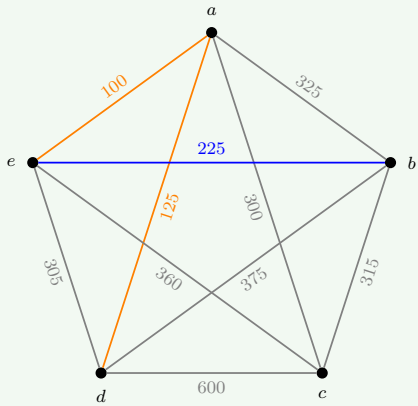


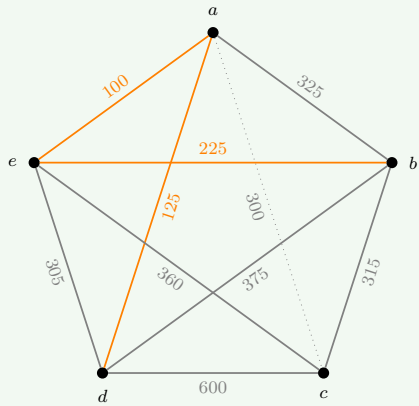




- Step 1. The smallest weight is $100$ for edge $ae$
- Step 2. The next smallest weight is $125$ with edge $ad$

# Cheapest Link Algorithm – example

## Example

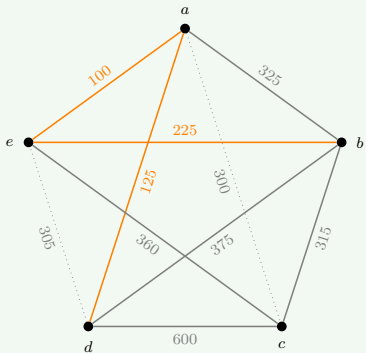


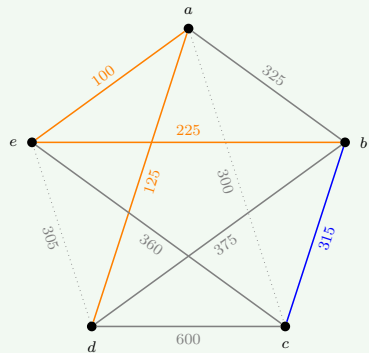




- Step 2. The next smallest weight is $225$ for $be$
- Step 2. Edge $ac$ has the next smallest weight, but choosing it would give vertex $a$ three highlighted incident edges.

# Cheapest Link Algorithm – example

## Example

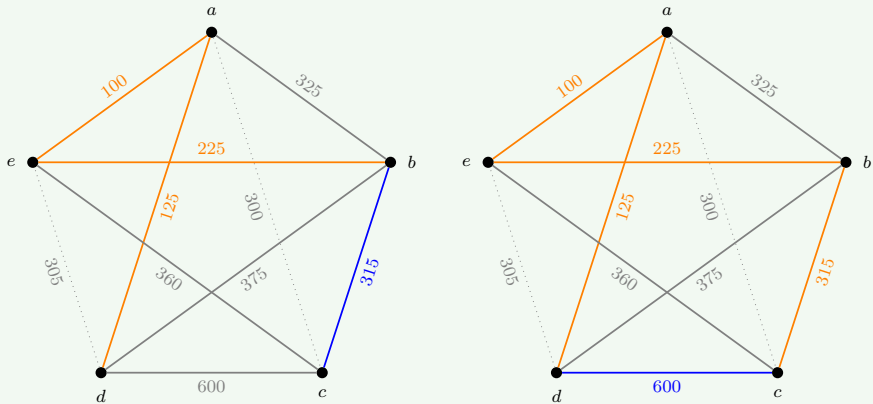



- Step 2. Edge $ed$ has the next smallest weight, but choosing it would create the smaller cycle $eade$ before all vertices are included, and it would also give vertex $e$ three highlighted incident edges.
- Step 2. The next available edge is $bc$.

# Cheapest Link Algorithm – example

## Example




- Step 2. At this point, we must close the cycle, and the only possible choice is $cd$.
- Output: The resulting cycle is $aebcda$, with total weight $1365$.

# Remarks

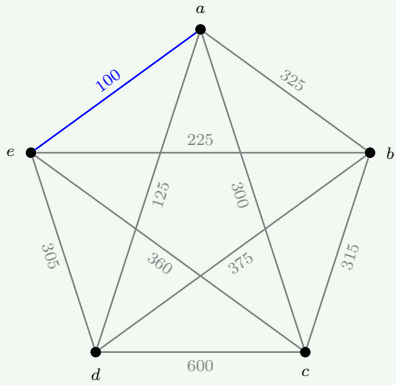
- In the example above, Cheapest Link ran into the same trouble as the initial cycle created using Nearest Neighbor
- Although the lightest edges were chosen, the heaviest also had to be included due to the outcome of the previous steps
- The downfall of the last two algorithms is the focus on choosing the smallest weighted edges at every opportunity.
- In some instances, it may be beneficial to work for a balance —choose more moderate weight edges to avoid later using the heaviest.
- Nearest Insertion Algorithm: forms a small circuit initially from the smallest weighted edge and balances newly added edges with the removal of a previously chosen edge.

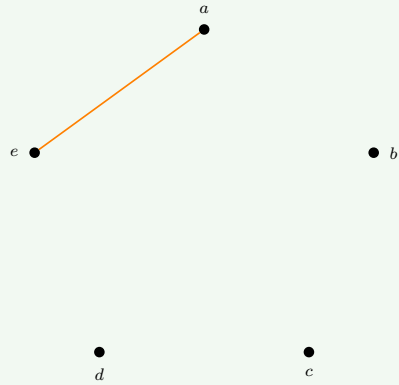
# Nearest Insertion Algorithm

1. Choose an edge of smallest weight. Highlight the edge and its endpoints.
2. Choose a vertex closest to one of the two highlighted vertices. Highlight this new vertex and the two edges joining it to the highlighted vertices.
3. Identify the unvisited vertex $u$ closest to the current cycle, and a closest vertex $v$ on that cycle. Insert $u$ next to $v$ by replacing one of the two cycle edges incident to $v$. Choose the replacement that gives the smaller increase in total weight.
4. Repeat Step 3 until all vertices have been included in the cycle.
5. Calculate the total weight.

# Nearest Insertion Algorithm – example

## Example

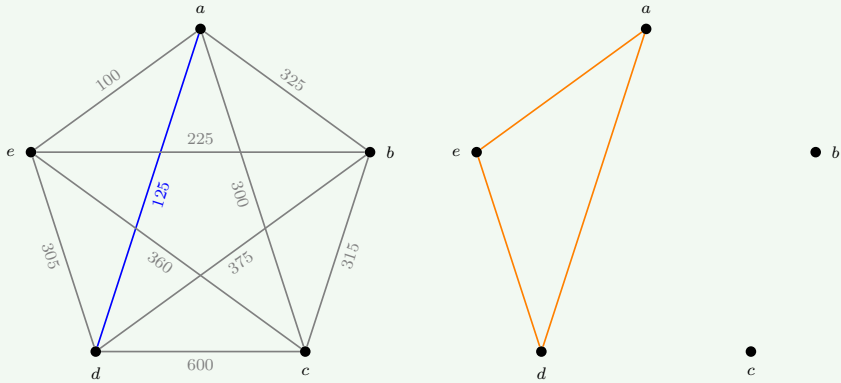



- Step 1. The smallest-weight edge is $ae$.

# Nearest Insertion Algorithm – example

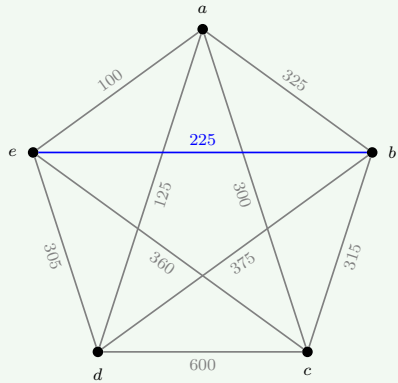
## Example

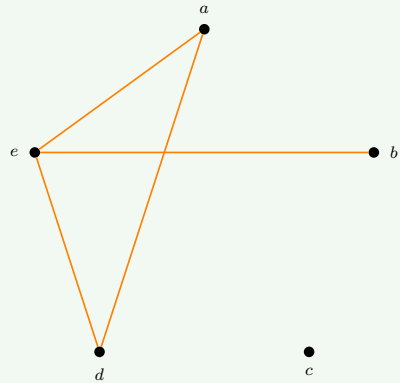



- Step 2: The closest vertex to either $a$ or $e$ is $d$, via edge $ad$. Form a cycle by adding $ad$ and $de$.
- Step 3: The closest vertex to one of $a$, $d$, or $e$ is $b$, via edge $be$.

# Nearest Insertion Algorithm – example

## Example



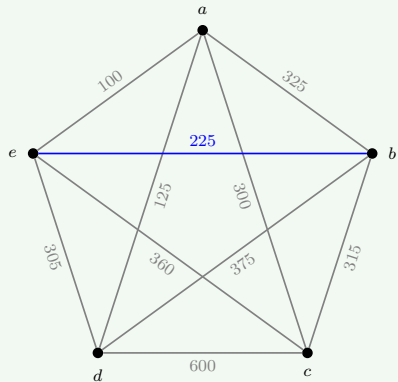



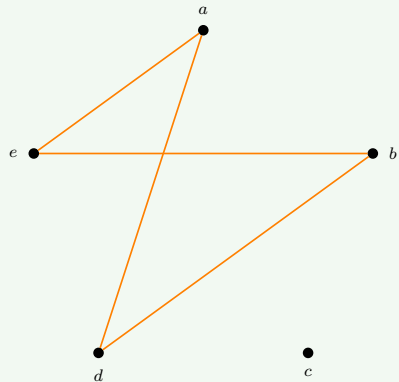

Step 3: After adding edge $be$, we must decide whether to add $ba$ and delete $ea$, or add $bd$ and delete $ed$.

$$\begin{aligned} \omega(ba) - \omega(ea) &= 325 - 100 = 225 \\ \omega(bd) - \omega(ed) &= 375 - 305 = \mathbf{70} \end{aligned}$$

# Nearest Insertion Algorithm – example

## Example

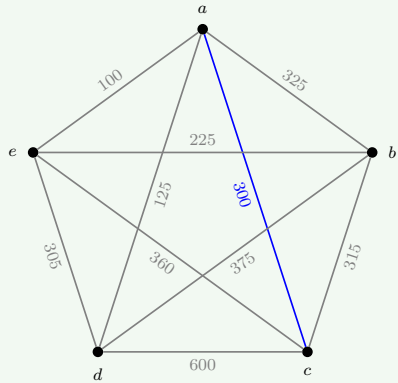


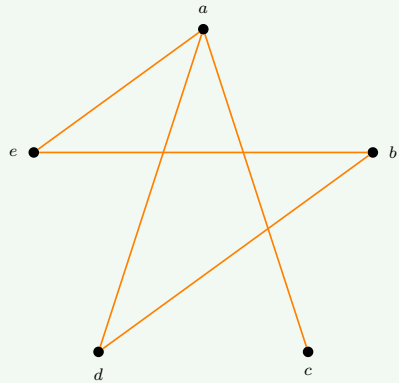




- Step 3: We add edge $bd$ and remove edge $ed$ to obtain a larger cycle.
- Step 3: The only remaining vertex is $c$, and $c$ is closest to $a$.

# Nearest Insertion Algorithm – example

## Example



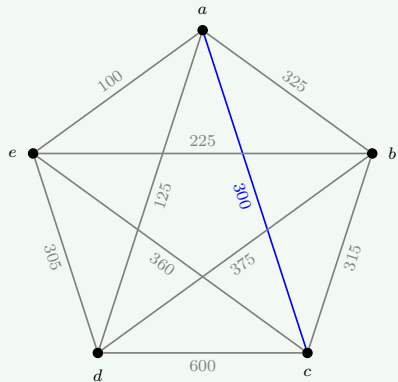



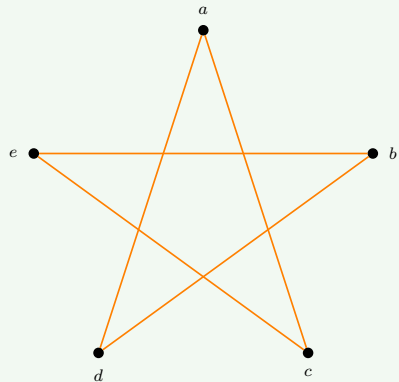

Step 3: After adding edge $ca$, we must decide whether to add $cd$ and delete $ad$, or add $ce$ and delete $ae$.

$$\begin{aligned} \omega(cd) - \omega(ad) &= 600 - 125 = 475 \\ \omega(ce) - \omega(ae) &= 360 - 100 = \mathbf{260} \end{aligned}$$

# Nearest Insertion Algorithm – example

## Example




- Step 3: We add edge $ce$ and remove edge $ae$ to obtain a larger cycle.
- Step 5: The resulting cycle is $acebda$, with total weight $1385$.

# Eulerian tours and Hamiltonian cycles

- Definitions and terminologies
- Different types of graphs
- Touring a graph
- Eulerian circuit algorithms
- Eulerization
- Existence of Hamiltonian cycles
- Traveling salesman problem
- Digraphs and Asymmetric Traveling Salesman Problem

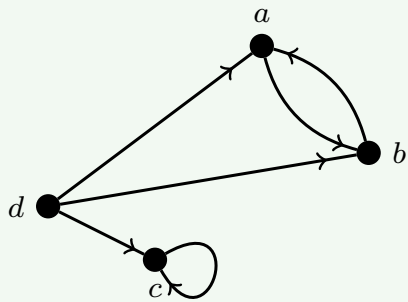
# Directed graph

## Definition

A *directed graph*, or *digraph*, is a graph $G = (V, A)$ that consists of a vertex set $V$ and an *arc set* $A$. An *arc* is an ordered pair of vertices.

## Example

Let $G$ be a digraph with $V = \{a, b, c, d\}$ and $A = \{ab, ba, cc, dc, db, da\}$.



# Terminologies

## Definition

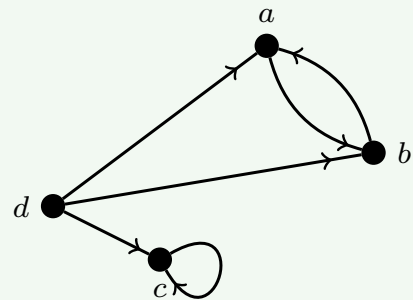
Let $G = (V, A)$ be a digraph

- Arc $yx$, $x$ is *head*, $y$ is *tail*
- *in-degree* of vertex $x$: the number of arcs for which $x$ is a head, denoted $\deg^-(x)$
- *out-degree* of vertex $x$: the number of arcs for which $x$ is the tail, denoted $\deg^+(x)$
- The *underlying graph* for a digraph is the graph $G' = (V, E)$ which is formed by removing the direction from each arc to form an edge
- A *directed path* is a path in which the head of an arc is the tail of the next arc in the path
- A *directed cycle* is a cycle in which the head of an arc is the tail of the next arc in the cycle

# Terminologies – example

## Example

- $a$ is the tail of arc $ab$ and the head of arcs $da$ and $ba$
- $\deg^-(a) = 2$, $\deg^-(b) = 2$, $\deg^-(c) = 2$, $\deg^-(d) = 0$
- $\deg^+(a) = 1$, $\deg^+(b) = 1$, $\deg^+(c) = 1$, $\deg^+(d) = 3$
- $d \to a \to b$ is a directed path
- $c \to d \to a \to b$ is not a directed path since $cd$ is not an arc in the graph
- $a \to b \to a$ is a directed cycle
- $d \to a \to b \to d$ is not a directed cycle



# Degrees of vertices

**Theorem**

*Let $G = (V, A)$ be a finite digraph[a], and $|A|$ denote the number of arcs in $G$. Both the sum of the in-degrees and the sum of the out-degrees of all vertices equal the number of arcs. In other words, if $V = \{v_1, v_2, \ldots, v_n\}$, then*

$$\deg^-(v_1) + \cdots + \deg^-(v_n) = |A| = \deg^+(v_1) + \cdots + \deg^+(v_n)$$

[a] both $|V|$ and $|A|$ are finite

- Since each arc will contribute to the in-degree to its head and to the out-degree of its tail, the sum of the in-degrees equals the sum of the out-degrees.
- Each arc is counted exactly once in the sum of the in-degrees (or out-degrees) since it has a unique head (or tail).

# Hamiltonian cycles

A few properties should be immediately apparent for a digraph to have a Hamiltonian cycle

- The underlying graph of $G$ must be connected.
- No vertex of $G$ can have in-degree or out-degree $0$
- $G$ cannot contain a cut-vertex

# A sufficient condition for Hamiltonian cycles

**Theorem**

*Let $G$ be a digraph with $n \geq 3$ vertices and no loops. If*

$$\deg^{-}(v) \geq \frac{n}{2}, \quad \deg^{+}(v) \geq \frac{n}{2}$$

*for every vertex $v$ of $G$, then $G$ has a Hamiltonian cycle.*

# Asymmetric Traveling Salesman Problem

- The Asymmetric Traveling Salesman Problem is to find the optimal Hamiltonian *directed* cycle on a digraph.
- The weight of the arc $ab$ need not equal the weight of arc $ba$
- *Complete digraph*: for each pair of distinct vertices $x$ and $y$ both arcs $xy$ and $yx$ exist in the graph
- We will only consider complete digraphs in which each arc has a positive weight
- To utilize the algorithms discussed before, we convert a digraph into an undirected graph

# Undirecting Algorithm

Input: Weighted complete digraph $G = (V, A, \omega)$
Steps:

1. For each vertex $x$, create a clone $x'$. Form the edge $xx'$ with weight $0$.
2. For each arc $xy$, form the edge $x'y$.
3. Assign each edge the weight of its corresponding arc: $\omega(x'y) = \omega(xy)$.

Output: weighted clone graph $G' = (V', E, \omega')$

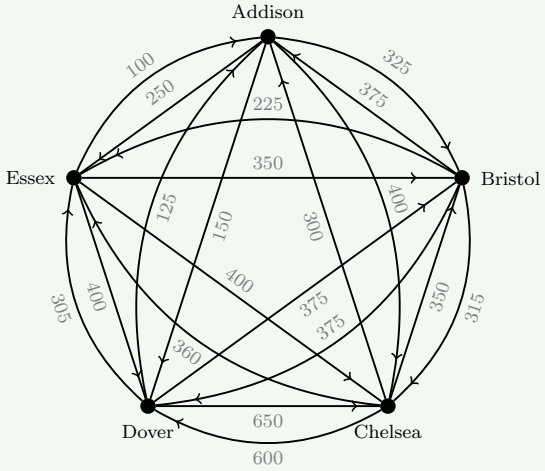
**Note**

- We need to ensure that when forming a cycle an arc into a vertex is immediately followed by an arc out of that same vertex
- By giving the edge between a vertex and its clone a weight of $0$ (when all other arcs have positive weights), we ensure these edges will be selected by the Nearest Neighbor and Cheapest Link Algorithms.
- In addition, this extra edge does not impact the total weight of the final cycle

# Undirecting Algorithm – example

## Example

Liz is heading out to visit her customers, and the direction of travel affects the cost.



# Undirecting Algorithm – example

## Example

Table representation. e.g. Addison to Bristol - 325

| | Addison | Bristol | Chelsea | Dover | Essex |
|---|---|---|---|---|---|
| Addison | . | 325 | 400 | 150 | 250 |
| Bristol | 375 | . | 315 | 375 | 225 |
| Chelsea | 300 | 350 | . | 600 | 360 |
| Dover | 125 | 375 | 650 | . | 305 |
| Essex | 100 | 350 | 400 | 400 | . |

# Undirecting Algorithm – example

## Example

Full weight matrix: symmetric because the graph is undirected; upper-right block is the transpose of the lower-left block; the other two blocks contain no edges.

| | a | b | c | d | e | a' | b' | c' | d' | e' |
|---|---|---|---|---|---|---|---|---|---|---|
| a | . | 325 | 400 | 150 | 250 | 0 | 375 | 300 | 125 | 100 |
| b | 375 | . | 315 | 375 | 225 | 325 | 0 | 350 | 375 | 350 |
| c | 300 | 350 | . | 600 | 360 | 400 | 315 | 0 | 650 | 400 |
| d | 125 | 375 | 650 | 0 | 305 | 150 | 375 | 600 | 0 | 400 |
| e | 100 | 350 | 400 | 400 | 0 | 250 | 225 | 360 | 305 | 0 |
| a' | 0 | 325 | 400 | 150 | 250 | . | . | . | . | . |
| b' | 375 | 0 | 315 | 375 | 225 | . | . | . | . | . |
| c' | 300 | 350 | 0 | 600 | 360 | . | . | . | . | . |
| d' | 125 | 375 | 650 | 0 | 305 | . | . | . | . | . |
| e' | 100 | 350 | 400 | 400 | 0 | . | . | . | . | . |

# Undirecting Algorithm – example

## Example

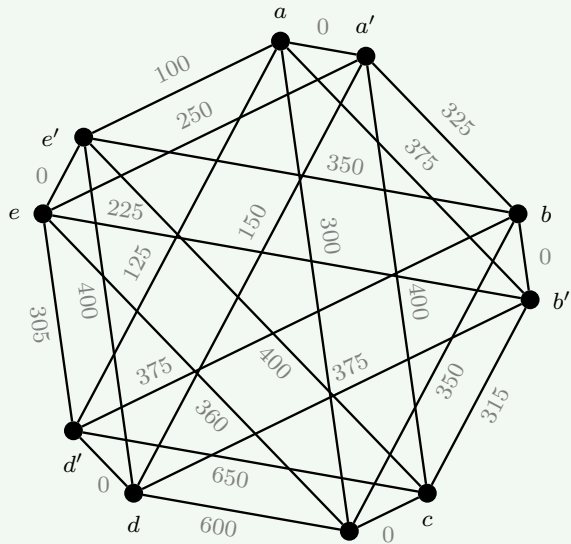
The weights of the clone graph can be specified using only the following table.

| | $a$ | $b$ | $c$ | $d$ | $e$ |
|---|---|---|---|---|---|
| $a'$ | 0 | 325 | 400 | 150 | 250 |
| $b'$ | 375 | 0 | 315 | 375 | 225 |
| $c'$ | 300 | 350 | 0 | 600 | 360 |
| $d'$ | 125 | 375 | 650 | 0 | 305 |
| $e'$ | 100 | 350 | 400 | 400 | 0 |

Since every edge connects an original vertex to a clone, this table contains all the edge weights of the clone graph.

# Undirecting Algorithm – example

## Example



# Remarks

- Even though the input to the algorithm is a complete digraph, the output is not a complete graph.
- The Undirecting Algorithm can be applied to a digraph that is not complete; however, the resulting graph may not have a Hamiltonian cycle.
- When applied to a complete digraph $G$ with $n$ vertices, the resulting graph $G'$ has $2n$ vertices, each of degree $n$, is guaranteed to have a Hamiltonian cycle.
- The cycle from $G'$ can then be translated to a Hamiltonian directed cycle of the digraph $G$ where vertex copies get reduced back to a single vertex

# Asymmetric Traveling Salesman Problem – example

## Example

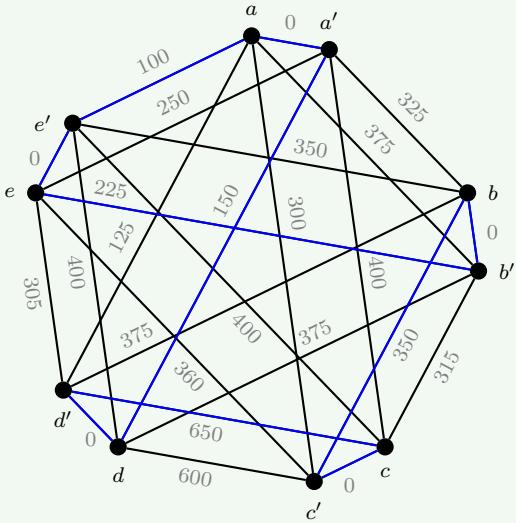
Continue from the previous example, the table below lists four cycles found using Nearest Neighbor and their conversion to a directed cycle in the digraph.

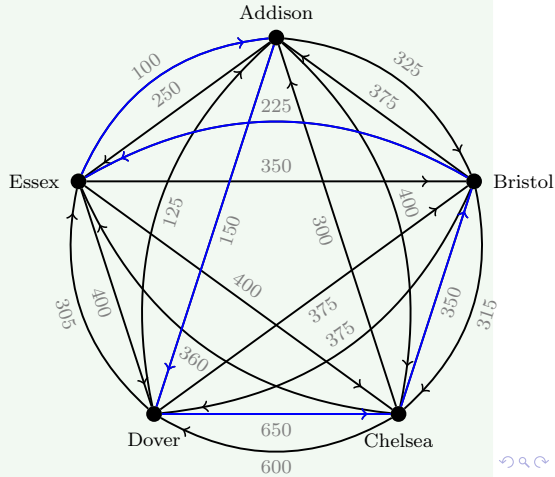
| Nearest Neighbor Cycle | Conversion | Total Weight |
|---|---|---|
| $a\ a'\ d\ d'\ e\ e'\ b\ b'\ c\ c'\ a$ | $a \to d \to e \to b \to c \to a$ | 1420 |
| $a'\ a\ e'\ e\ b'\ b\ c'\ c\ d'\ d\ a'$ | $a \to d \to c \to b \to e \to a$ | 1475 |
| $e\ e'\ a\ a'\ d\ d'\ b\ b'\ c\ c'\ e$ | $e \to a \to d \to b \to c \to e$ | 1300 |
| $e'\ e\ b'\ b\ c'\ c\ a'\ a\ d'\ d\ e'$ | $e \to d \to a \to c \to b \to e$ | 1500 |

Note that cycles beginning with a clone vertex must be reversed when translating them back into a directed cycle.

# Asymmetric Traveling Salesman Problem – example

## Example



# Remarks

- Unlike in the undirected case, reversals of a Hamiltonian cycle in a digraph (that is not a complete digraph) might not exist, and those that do will most likely result in a different total weight.
- Thus when applying the Repetitive Nearest Neighbor Algorithm to graphs formed using the Undirecting Algorithms, we must consider starting at both copies of a vertex, such as $a$ and $a'$, $e$ and $e'$ as in the example

# Cheapest Link Algorithm

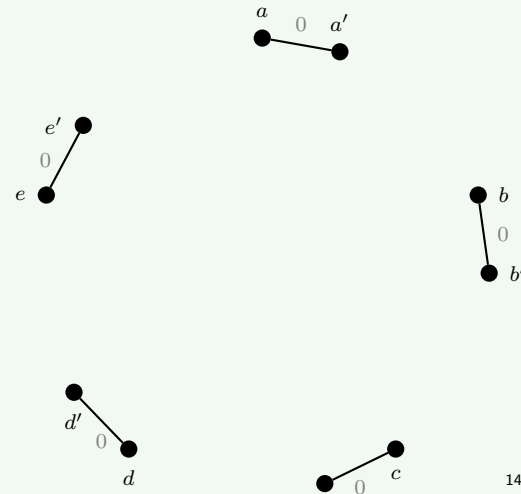
- All edges of weight $0$ must be included in the final cycle since picking the edges of minimum weight would initially result in choosing all the weight $0$ edges as no two of these are adjacent (so there is no concern of a vertex having degree $3$ or closing the circuit too early)

# Cheapest Link Algorithm – example

## Example

Continuing from the previous example.

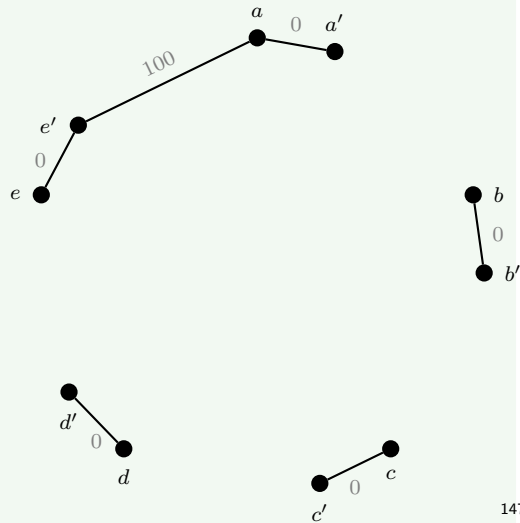
- Step 1. Since all edges between a vertex and its clone have a weight of $0$, all of these edges will be chosen



# Cheapest Link Algorithm – example

## Example

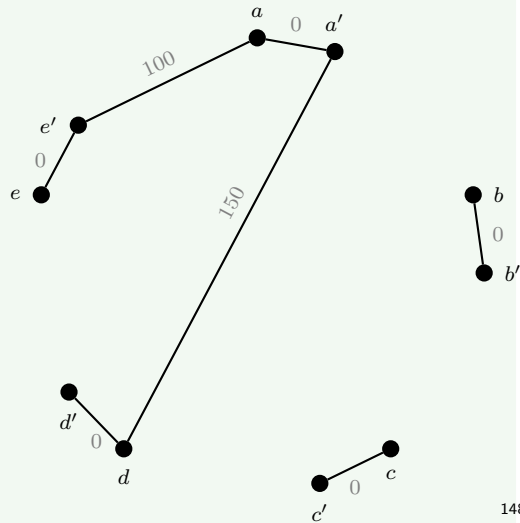
- Step 2. The edge of smallest weight is $ae'$



# Cheapest Link Algorithm – example

## Example

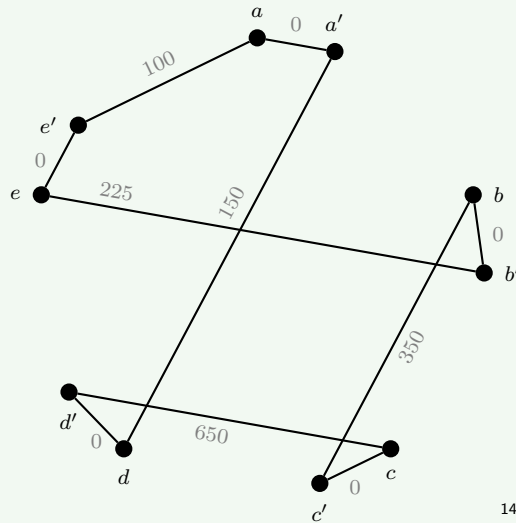
- Step 2. The next lowest is $ad'$. However, we cannot choose this edge since it would cause $a$ to be incident to three highlighted edges. We skip this edge and move to $a'd$.



# Cheapest Link Algorithm – example

## Example

- Step 2. Continuing with the Algorithm, we add edges $b'e$, $c'b$. Then we close the cycle with edge $cd'$.

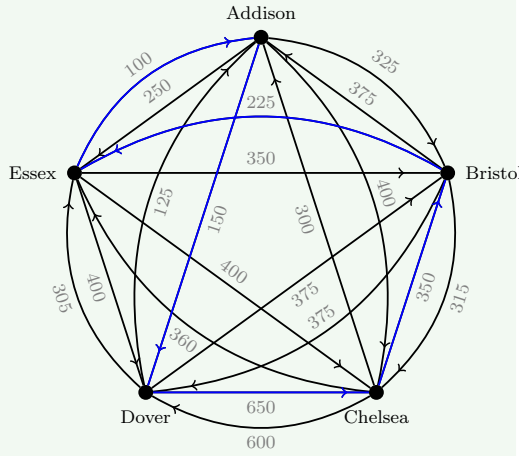


# Cheapest Link Algorithm – example

## Example

- Step 2. Convert the cycle $aa'dd'cc'bb'ee'a$ to directed cycle in the digraph

$$a \to d \to c \to b \to e \to a$$



# Nearest Insertion Algorithm

- The Nearest Insertion Algorithm, in its original form, does not work on weighted clone graphs.
- There are no $3$-cycles in the graph produced by the Undirecting Algorithm, but Step 2 of Nearest Insertion requires a $3$-cycle built from the cheapest edge.
- A modified version of Nearest Insertion for weighted clone graphs, which treats the weight-$0$ edges differently, will be discussed in the tutorial.

# About tutorials

- Enrichment Questions
  - These questions are intended to deepen your understanding.
  - Will *NOT* appear in the exams
  - Participation is optional and for your interest.
  - If you'd like to discuss them, feel free to email me or your tutor (cvičiaci) to arrange a meeting.